HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BÀI TẬP NHÓM

ĐỀ TÀI: Xây dựng hệ cơ sở dữ liệu ứng dụng nghe nhạc online

VŨ MINH SÁNG	MSV: B23DCCE082
NGUYỄN HỒNG QUANG	MSV: B23DCVT361
AN VIỆT QUYỀN	MSV: B23DCVT365
VŨ VIỆT THÁI	MSV: B23DCCE085
ĐỖ TIẾN THÀNH	MSV: B23DCDT239

GIẢNG VIÊN HƯỚNG DẪN: TS.NGUYỄN THỊ THANH THỦY

Hà Nội - 2026

PHÂN CHIA CÔNG VIỆC

Công việc chung của nhóm
Khảo sát và Phân tích yêu cầu nghiệp vụ: Cùng nhau thảo luận, bóc tách các quy trình thực tế của hệ thống đào tạo tín chỉ tại trường đại học (bao gồm quy chế mở lớp, xếp lịch học, đăng ký môn tiên quyết và đối soát học phí). **Thiết kế Mô hình Khái niệm (ERD) và Mô hình Quan hệ (RM)**: Tổ chức các buổi họp nhóm để chốt danh sách 13 thực thể (Entities) và thống nhất các mối quan hệ (1-N). Đây là bước quan trọng nhất yêu cầu sự tham gia của cả nhóm để giải quyết các luồng dữ liệu khó như: rã các quan hệ Nhiều - Nhiều (M-N) thành các bảng giao dịch độc lập, quyết định thiết lập khóa chính đơn thay cho khóa kép, và tối ưu hóa các thực thể phụ thuộc. **Review và chuẩn hóa CSDL**: Cùng nhau rà soát lại Mô hình quan hệ (RM) để đảm bảo 100% các bảng đều đạt Dạng chuẩn 3 (3NF), không bị dư thừa hoặc dị thường dữ liệu trước khi bắt tay vào gõ code SQL. **Tổng hợp báo cáo và củng cố kiến thức**: Cùng nhau rà soát lỗi trình bày, đối chiếu sự đồng nhất tuyệt đối giữa phần lý thuyết (file Word) và phần thực hành (file Code SQL). Đặc biệt, các thành viên tổ chức review chéo, hỗ trợ và chia sẻ kiến thức cho nhau để đảm bảo cả nhóm đều nắm vững bản chất luồng dữ liệu, tự tin chuẩn bị cho buổi báo cáo.

Họ và tên	Mã sinh viên	Nội dung được giao	Đóng góp
Nguyễn Hồng Quang	B23DCVT361	- Đề xuất và cài đặt chiến lược Đánh chỉ mục (Indexing) với các phần/câu hỏi được giao - Hỗ trợ viết và tối ưu hóa mã lệnh SQL.	14%
An Việt Quyền	B23DCVT365	- Đề xuất và cài đặt chiến lược Đánh chỉ mục (Indexing) với các phần/câu hỏi được giao - Hỗ trợ viết và tối ưu hóa mã lệnh SQL.	20%
Vũ Minh Sáng	B23DCCE082	- Đề xuất và cài đặt chiến lược Đánh chỉ mục (Indexing) với các phần/câu hỏi được giao. - Hỗ trợ viết và tối ưu hóa mã lệnh SQL - Phát triển ứng dụng Web Demo (Giao diện & Kết nối Database). - Hỗ trợ triển khai vật lý trên SQL	24%

		Server (DDL) và nạp dữ liệu mẫu (DML). - Xây dựng các biểu thức Đại số quan hệ.	
Vũ Việt Thái	B23DCCE085	- Đề xuất và cài đặt chiến lược Đánh chỉ mục (Indexing) với các phần/câu hỏi được giao - Hỗ trợ viết và tối ưu hóa mã lệnh SQL.	20%
Đỗ Tiến Thành	B23DCDT239	- Hỗ trợ triển khai vật lý trên SQL Server (DDL) và nạp dữ liệu mẫu (DML). - Đề xuất và cài đặt chiến lược Đánh chỉ mục (Indexing) với các phần/câu hỏi được giao - Hỗ trợ viết và tối ưu hóa mã lệnh SQL.	22

LỜI NÓI ĐẦU

Trong kỷ nguyên số hóa hiện nay, nhu cầu giải trí trực tuyến, đặc biệt là thưởng thức âm nhạc, đã trở thành một phần thiết yếu trong đời sống tinh thần của con người. Sự bùng nổ của các nền tảng như Spotify, Apple Music hay YouTube Music đã minh chứng cho sức mạnh của việc kết nối dữ liệu âm nhạc khổng lồ với trải nghiệm cá nhân hóa của người dùng. Đứng sau sự vận hành mượt mà của những hệ thống này chính là một cấu trúc cơ sở dữ liệu (CSDL) đồ sộ, khoa học và tối ưu.

Nhận thức được tầm quan trọng đó, nhóm chúng em đã lựa chọn đề tài: **"Xây dựng hệ cơ sở dữ liệu ứng dụng nghe nhạc online"** cho bài tập lớn môn Hệ Quản trị cơ sở dữ liệu. Dự án không chỉ dừng lại ở việc lưu trữ thông tin bài hát đơn thuần mà còn hướng tới một hệ sinh thái quản lý toàn diện.

Bản báo cáo này tập trung vào việc giải quyết các bài toán nghiệp vụ thực tế như:

- **Quản lý tài nguyên âm nhạc đa tầng**: Phân cấp rõ ràng từ Nghệ sĩ, Album đến Bài hát, hỗ trợ đa nghệ sĩ và đa thể loại.
- **Cá nhân hóa và tương tác**: Hệ thống hóa danh sách phát, lịch sử nghe nhạc, cùng các tính năng cộng đồng như bình luận và yêu thích.
- **Vận hành dịch vụ và kinh doanh**: Thiết lập mô hình quản lý gói đăng ký và lịch sử thanh toán, đảm bảo tính bền vững cho nền tảng.

Với mục tiêu tối ưu hóa hiệu năng và đảm bảo tính toàn vẹn dữ liệu, nhóm đã nỗ lực thiết kế mô hình quan hệ đạt dạng chuẩn BCNF , đồng thời cài đặt các ràng buộc chặt chẽ và chiến lược đánh Index để đáp ứng các truy vấn phức tạp trên quy mô lớn.

Chúng em xin chân thành cảm ơn **TS. Nguyễn Thị Thanh Thủy** đã tận tình hướng dẫn và đưa ra những góp ý quý báu để nhóm hoàn thiện dự án này. Dù đã có nhiều cố gắng, song bản báo cáo khó tránh khỏi những thiếu sót, kính mong nhận được sự phê bình và đóng góp từ cô để kiến thức của chúng em ngày một hoàn thiện hơn.

I. Kịch bản thế giới thực.....5
1. Ứng dụng của hệ cơ sở dữ liệu.....5
2. Các nội dung của CSDL.....5
3. Các thao tác trên cơ sở dữ liệu.....6
4. Các yêu cầu về dữ liệu cần được lưu trữ.....6
II. Xác định thực thể & quy tắc.....7
1. Xác định thực thể, thuộc tính.....7
2. Các quy tắc nghiệp vụ.....7
III. Mô hình E-R.....8
IV. Chuẩn hóa và chuyển mô hình ER sang mô hình quan hệ.....9
A. Ánh xạ về các dạng chuẩn.....9
B. Xác định phụ thuộc hàm và khóa.....11
C. Chuẩn hóa.....14
V. Phân tích ràng buộc.....17
1. Ràng buộc miền giá trị (Domain Constraints).....17
2. Ràng buộc toàn vẹn tham chiếu (Referential Integrity).....18
VI. Cài đặt CSDL.....20
1. Cài đặt bảng.....20
2. Dữ liệu mẫu được nhập vào.....26
VII. Thử nghiệm với CSDL đã thiết kế.....28
VIII. Tối ưu hóa hiệu năng truy vấn với Chỉ mục (Indexing).....41
1. Kịch bản 1: Tìm kiếm dữ liệu văn bản (Text Search).....41
2. Kịch bản 2: Truy vấn thống kê trên tập dữ liệu lớn (Aggregation).....42
3. Kịch bản 3: Truy vấn kết hợp nhiều điều kiện lọc.....42
4. Kịch bản 4: Tối ưu hóa truy vấn kết hợp sắp xếp.....42
IX. Kết luận.....43

I. Kịch bản thế giới thực

1. Ứng dụng của hệ cơ sở dữ liệu

- **Quản lý thông tin bài hát có hệ thống**: Lưu trữ phân cấp rõ ràng theo Nghệ sĩ → Album → Bài hát, hỗ trợ đa nghệ sĩ (featured, composer, producer) và phân loại theo thể loại.
- **Cá nhân hóa trải nghiệm**: Cho phép người dùng tạo và lưu trữ các Playlist cá nhân, giúp việc thưởng thức âm nhạc trở nên liền mạch và thuận tiện.
- **Quản lý dịch vụ & thanh toán**: Hỗ trợ các gói đăng ký (Free, Premium, Family) với các quyền lợi khác nhau, ghi nhận lịch sử thanh toán và trạng thái giao dịch.
- **Tương tác cộng đồng**: Người dùng có thể bình luận về bài hát, thả tim bình luận và yêu thích bài hát.
- **Phân quyền hệ thống**: Phân loại người dùng theo vai trò (admin, user, moderator) với mức quyền hạn khác nhau.

2. Các nội dung của CSDL

a) Dữ liệu tài nguyên bài hát

- **Thông tin nghệ sĩ**: Thông tin ca sĩ, nhạc sĩ sáng tác, quốc gia, tiểu sử.
- **Thông tin album**: Lưu trữ các album, năm phát hành, nhãn đĩa, giúp phân loại bài hát theo giai đoạn sáng tác của nghệ sĩ.
- **Thông tin bài hát**: Lưu trữ metadata của từng bài hát (tên bài, thời lượng, đường dẫn file, lời bài hát) và liên kết với album.
- **Thể loại âm nhạc**: Phân loại bài hát theo thể loại (Pop, Rock, Kpop, Vpop, Rap…).
- **Vai trò nghệ sĩ trong bài hát**: Nghệ sĩ chính (main), khách mời (featured), nhạc sĩ (composer), nhà sản xuất (producer).

b) Dữ liệu người dùng

- **Thông tin người dùng**: Quản lý thông tin định danh (tên người dùng, email, mật khẩu băm) để cấp quyền truy cập.
- **Vai trò & phân quyền**: Phân loại người dùng theo vai trò (admin/user/moderator) để quản lý quyền hạn.
- **Thông tin đăng ký dịch vụ**: Lưu trữ lịch sử đăng ký gói, thời hạn, trạng thái, tự động gia hạn.
- **Thông tin các danh sách phát**: Lưu trữ các bộ sưu tập nhạc do người dùng tự tạo.

c) Dữ liệu hành vi & tương tác

- **Lịch sử nghe nhạc**: Ghi nhận chi tiết từng lần người dùng bấm nghe một bài hát.
- **Lịch sử thanh toán**: Ghi nhận các giao dịch thanh toán, trạng thái, mã giao dịch.
- **Bình luận**: Bình luận bài hát, gắn nhãn thời gian trong bài hát.
- **Lượt thích**: Người dùng thích bài hát, thích bình luận.

3. Các thao tác trên cơ sở dữ liệu

- Thêm, sửa, xóa thông tin nhạc (bài hát, album, nghệ sĩ, thể loại)
- Thêm, sửa, xóa thông tin người dùng, phân quyền theo vai trò
- Quản lý gói đăng ký và thanh toán
- Tạo và quản lý danh sách nhạc cá nhân
- Tra cứu, tìm các bài hát theo tên, nghệ sĩ, album hoặc thể loại
- Ghi nhận lịch sử nghe nhạc; thống kê bài hát phổ biến
- Bình luận, thả tim bài hát và bình luận

4. Các yêu cầu về dữ liệu cần được lưu trữ

- **Dữ liệu vai trò (ROLES):**
  + Mã vai trò (Thuộc tính khóa, duy nhất)
  + Tên vai trò (admin / user / moderator)
  + Mô tả quyền hạn
- **Dữ liệu người dùng (USERS):**
  + Mã người dùng (Thuộc tính khóa, duy nhất)
  + Tên người dùng - Đường dẫn ảnh đại diện
  + Mật khẩu băm
  + Email
  + Thời gian được tạo
  + Vai trò (liên kết ROLES)
- **Dữ liệu gói đăng ký (SUBSCRIPTION_PLANS):**
  + Mã gói (Thuộc tính khóa, duy nhất)
  + Tên gói (Free / Premium / Family)
  + Giá gói, thời hạn (tháng)
  + Quyền lợi: không quảng cáo, âm thanh cao cấp, số bài tải về
- **Dữ liệu đăng ký (USER_SUBSCRIPTIONS):**
  + Mã đăng ký (Thuộc tính khóa, duy nhất)
  + Người dùng và gói đăng ký
  + Ngày bắt đầu, ngày kết thúc, trạng thái, tự động gia hạn
- **Dữ liệu thanh toán (PAYMENTS):**
  + Mã thanh toán (Thuộc tính khóa, duy nhất)
  + Mã giao dịch (duy nhất)
  + Số tiền, thời điểm thanh toán, trạng thái
- **Dữ liệu nghệ sĩ (ARTISTS):**
  + Mã nghệ sĩ (Thuộc tính khóa, duy nhất)
  + Tên nghệ sĩ, quốc gia, tiểu sử ngắn, đường dẫn ảnh nền
- **Dữ liệu album (ALBUMS):**
  + Mã album (Thuộc tính khóa, duy nhất)
  + Tên album, năm ra mắt, đường dẫn ảnh bìa, nhãn đĩa, nghệ sĩ tạo ra album
- **Dữ liệu bài hát (SONGS):**
  + Mã bài (Thuộc tính khóa, duy nhất)
  + Tên bài hát, đường dẫn file, thời lượng, album, thời gian tải lên, lời bài hát

- **Dữ liệu thể loại (GENRES):**
    + Mã thể loại (Thuộc tính khóa, duy nhất)
    + Tên thể loại, mô tả thể loại
- **Dữ liệu danh sách phát (PLAYLISTS):**
    + Mã danh sách phát (Thuộc tính khóa, duy nhất)
    + Tên, mô tả, trạng thái (công khai / riêng tư)
    + Các bài hát bên trong (qua bảng trung gian PLAYLIST_SONGS)
    + Người dùng tạo ra, thời gian tạo và cập nhật
- **Dữ liệu lịch sử nghe (USER_SONGS):**
    + Mã lượt nghe (Thuộc tính khóa, duy nhất)
    + Người dùng, bài hát và thời điểm nghe

II. Xác định thực thể & quy tắc

1. Xác định thực thể, thuộc tính

- **ROLES** (role_id, role_name, permissions)
- **USERS** (user_id, username, email, password_hash, avatar_url, created_at, #role_id)
- **SUBSCRIPTION_PLANS** (plan_id, plan_name, price, duration_months, ad_free, high_quality_audio, max_downloads)
- **USER_SUBSCRIPTIONS** (subscription_id, #user_id, #plan_id, start_date, end_date, status, auto_renew)
- **PAYMENTS** (payment_id, #subscription_id, amount, payment_date, status, transaction_id)
- **ARTISTS** (artist_id, name, country, bio, cover_image_url)
- **ALBUMS** (album_id, title, #artist_id, release_year, cover_image_url, record_label)
- **SONGS** (song_id, name, #album_id, duration, file_path, lyrics, uploaded_at)
- **GENRES** (genre_id, genre_name, description)
- **PLAYLISTS** (playlist_id, #user_id, name, description, is_public, created_at, updated_at)
- **USER_SONGS** (listen_id, #user_id, #song_id, listened_at)

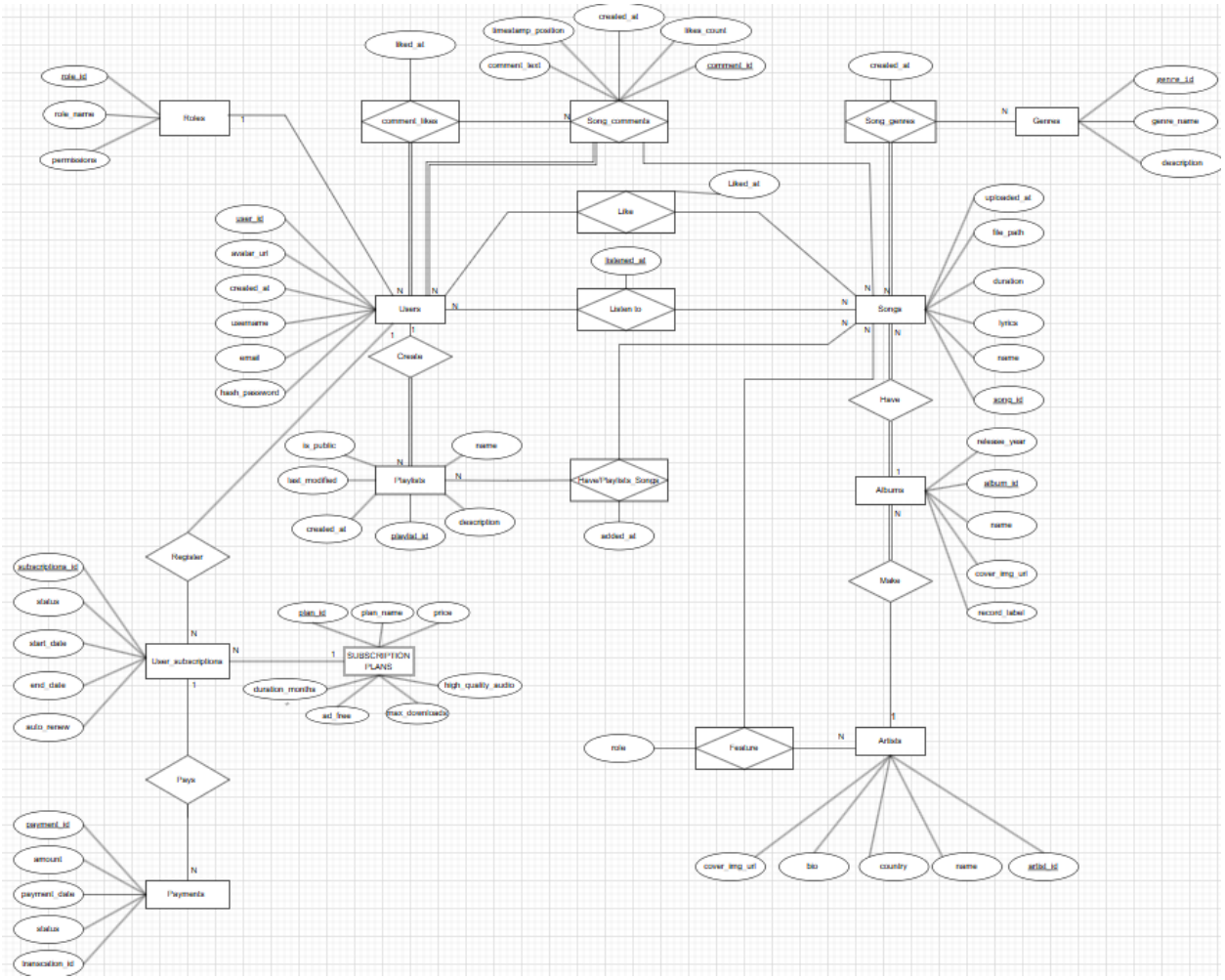
2. Các quy tắc nghiệp vụ

- Một user có một vai trò; một vai trò có thể gán cho nhiều user (1-N)
- Một user có thể đăng ký 1 hoặc nhiều gói dịch vụ theo thời gian (1-N)
- Một user có thể có 1 hoặc nhiều playlist (1-N)
- Một artist có thể phát hành 1 hoặc nhiều album (1-N)
- Một album có thể chứa một hoặc nhiều bài hát (1-N)
- Một bài hát có thể có nhiều nghệ sĩ tham gia với các vai trò khác nhau, một nghệ sĩ có thể tham gia nhiều bài hát (N-N) → SONG_ARTISTS
- Một bài hát có thể thuộc nhiều thể loại, một thể loại có thể gán cho nhiều bài hát (N-N) → SONG_GENRES
- Một playlist có thể chứa nhiều bài hát, một bài hát có thể ở trong nhiều playlist khác nhau (N-N) → PLAYLIST_SONGS

- Một user có thể nghe nhiều bài hát, một bài hát có thể được nghe bởi nhiều user nhiều lần (N-N) → USER_SONGS
- Người dùng có thể thích bài hát và thích bình luận (N-N) → SONG_LIKES, COMMENT_LIKES
- Khi một người dùng bị xóa, các playlist, lịch sử nghe, đăng ký dịch vụ của người đó cũng bị xóa (CASCADE)
- Khi một bài hát bị xóa, các bản ghi liên kết trong playlist sẽ bị xóa (CASCADE)
- Khi một album bị xóa, bài hát vẫn tồn tại nhưng album_id được đặt thành NULL (SET NULL)

III. Mô hình E-R

Mô hình E-R của hệ thống gồm các thực thể chính và mối quan hệ sau:



Link: [music.drawio]

Bảng tổng hợp quan hệ:

Thực thể A	Quan hệ	Thực thể B	Ghi chú
ROLES	1:N	USERS	Một vai trò cho nhiều users
USERS	1:N	USER_SUBSCRIPTIONS	Một user đăng ký nhiều gói
SUBSCRIPTION_PLANS	1:N	USER_SUBSCRIPTIONS	Một gói được nhiều user đăng ký
USER_SUBSCRIPTIONS	1:N	PAYMENTS	Một đăng ký có nhiều giao dịch
ARTISTS	1:N	ALBUMS	Một nghệ sĩ có nhiều album
ALBUMS	1:N	SONGS	Một album chứa nhiều bài hát
SONGS	N:N	ARTISTS	Qua bảng SONG_ARTISTS (có thuộc tính role)
SONGS	N:N	GENRES	Qua bảng SONG_GENRES
PLAYLISTS	N:N	SONGS	Qua bảng PLAYLIST_SONGS (có thuộc tính position)
USERS	N:N	SONGS	Qua bảng USER_SONGS (lịch sử nghe)
USERS	1:N	PLAYLISTS	Một user tạo nhiều playlist
USERS	1:N	SONG_COMMENTS	Một user có nhiều bình luận
SONGS	1:N	SONG_COMMENTS	Một bài hát có nhiều bình luận
USERS	N:N	SONG_COMMENTS	Qua bảng COMMENT_LIKES
USERS	N:N	SONGS	Qua bảng SONG_LIKES (lượt thích bài hát)

IV. Chuẩn hóa và chuyển mô hình ER sang mô hình quan hệ

A. Ánh xạ về các dạng chuẩn

Bảng ROLES

<u>role_id</u>	role_name	permissions

Bảng USERS

user_id	username	email	password_hash	avatar_url	created_at

Bảng SUBSCRIPTION_PLANS

plan_id	plan_name	price	duration_months	ad_free	high_quality_audio	max_downloads

Bảng ARTISTS

artist_id	name	country	bio	cover_image_url

Bảng ALBUMS

album_id	title	release_year	cover_image_url	record_label

Bảng SONGS

song_id	name	file_path	duration	lyrics	uploaded_at

Bảng GENRES

genre_id	genre_name	description

Bảng PLAYLISTS

playlist_id	name	description	is_public	created_at	updated_at

1. Chuyển đổi các liên kết 1:N

Quy tắc: "Bảng bên NHIỀU (N) phải chứa Khóa chính của bảng bên MỘT (1) để làm Khóa ngoại."

Dựa vào các quy tắc của CSDL đã đặt ra:

- Một user có một vai trò (1-N từ ROLES):

user_id	username	email	password_hash	avatar_url	created_at	**#role_id**

- Mỗi lần đăng ký gói thuộc về một user và một gói (1-N):

subscription_id	**#user_id**	**#plan_id**	start_date	end_date	status	auto_renew

- Mỗi thanh toán thuộc về một lần đăng ký (1-N):

payment_id	**#subscription_id**	amount	payment_date	status	transaction_id

- Một album thuộc về một nghệ sĩ (1-N từ ARTISTS):

album_id	title	**#artist_id**	release_year	cover_image_url	record_label

- Một bài hát thuộc về một album (1-N từ ALBUMS):

song_id	name	**#album_id**	duration	file_path	lyrics	uploaded_at

- Một playlist thuộc về một user (1-N từ USERS):

playlist_id	**#user_id**	name	description	is_public	created_at	updated_at

2. Chuyển đổi các liên kết N:N

Quy tắc: "Tách thành một bảng mới (Bảng trung gian). Khóa chính của bảng mới là tổ hợp khóa chính của 2 bảng kia."

Dựa vào các quy tắc CSDL đã đặt ra:

- Một bài hát có nhiều nghệ sĩ với vai trò khác nhau (N-N):

Bảng SONG_ARTISTS:

id	**#song_id**	**#artist_id**	role

- Một bài hát có thể thuộc nhiều thể loại (N-N):

Bảng SONG_GENRES:

id	**#song_id**	**#genre_id**

- Một playlist có thể chứa nhiều bài hát, một bài hát có thể ở trong nhiều playlist (N-N):

Bảng PLAYLIST_SONGS:

id	**#playlist_id**	**#song_id**	position	added_at

- Một user có thể nghe nhiều bài hát, một bài hát có thể được nghe bởi nhiều user nhiều lần:

Bảng USER_SONGS:

listen_id	**#user_id**	**#song_id**	listened_at

- Người dùng thích bài hát (N-N):

Bảng SONG_LIKES:

id	**#user_id**	**#song_id**	liked_at

- Người dùng thích bình luận (N-N):

Bảng COMMENT_LIKES:

id	**#user_id**	**#comment_id**	liked_at

B. Xác định phụ thuộc hàm và khóa

1. Bảng ROLES (**role_id**, role_name, permissions)

- Tập phụ thuộc hàm:
    - role_id → ALL
    - role_name → ALL (role_name là duy nhất)
- Các khóa dự tuyển: {role_id}, {role_name}
- Khóa chính: {role_id}

2. Bảng USERS (**<u>user_id</u>**, username, email, password_hash, avatar_url, created_at, **#role_id**)

- Tập phụ thuộc hàm:
    - user_id → ALL
    - username → ALL (username là duy nhất)
    - email → ALL (email là duy nhất)
- Các khóa dự tuyển: {user_id}, {username}, {email}
- Khóa chính: {user_id}

3. Bảng SUBSCRIPTION_PLANS (**<u>plan_id</u>**, plan_name, price, duration_months, ad_free, high_quality_audio, max_downloads)

- Tập phụ thuộc hàm: plan_id → ALL
- Các khóa dự tuyển: {plan_id}
- Khóa chính: {plan_id}

4. Bảng USER_SUBSCRIPTIONS (**<u>subscription_id</u>**, #user_id, #plan_id, start_date, end_date, status, auto_renew)

- Tập phụ thuộc hàm: subscription_id → ALL
- Các khóa dự tuyển: {subscription_id}
- Khóa chính: {subscription_id}

5. Bảng PAYMENTS (**<u>payment_id</u>**, **#subscription_id**, amount, payment_date, status, transaction_id)

- Tập phụ thuộc hàm:
    - payment_id → ALL
    - transaction_id → ALL (transaction_id là duy nhất)
- Các khóa dự tuyển: {payment_id}, {transaction_id}
- Khóa chính: {payment_id}

6. Bảng ARTISTS (**<u>artist_id</u>**, name, country, bio, cover_image_url)

- Tập phụ thuộc hàm: artist_id → ALL
- Các khóa dự tuyển: {artist_id}
- Khóa chính: {artist_id}

7. Bảng ALBUMS (**album_id**, title, **#artist_id**, release_year, cover_image_url, record_label)

- Tập phụ thuộc hàm: album_id → ALL
- Các khóa dự tuyển: {album_id}
- Khóa chính: {album_id}

8. Bảng SONGS (**song_id**, name, **#album_id**, duration, file_path, lyrics, uploaded_at)

- Tập phụ thuộc hàm:
  - song_id → ALL
  - file_path → ALL (file_path là duy nhất)
- Các khóa dự tuyển: {song_id}, {file_path}
- Khóa chính: {song_id}

9. Bảng GENRES (**genre_id**, genre_name, description)

- Tập phụ thuộc hàm:
  - genre_id → ALL
  - genre_name → ALL (genre_name là duy nhất)
- Các khóa dự tuyển: {genre_id}, {genre_name}
- Khóa chính: {genre_id}

10. Bảng PLAYLISTS (**playlist_id**, **#user_id**, name, description, is_public, created_at, updated_at)

- Tập phụ thuộc hàm: playlist_id → ALL
- Các khóa dự tuyển: {playlist_id}
- Khóa chính: {playlist_id}

11. Bảng SONG_ARTISTS (**id**, **#song_id**, **#artist_id**, role)

- Tập phụ thuộc hàm: song_id, artist_id → role
- Các khóa dự tuyển: {id, song_id, artist_id}
- Khóa chính: {id}

12. Bảng SONG_GENRES (**id**, **#song_id**, **#genre_id**)

- Tập phụ thuộc hàm:{id}, {song_id, genre_id} → ∅ (Tất cả thuộc tính nằm trong khóa)
- Các khóa dự tuyển: {id, song_id, genre_id}
- Khóa chính: {id}

13. Bảng PLAYLIST_SONGS (**id**, **#playlist_id**, **#song_id**, position, added_at)

- Tập phụ thuộc hàm: {id}, {playlist_id, song_id}→ position, added_at
- Các khóa dự tuyển: {playlist_id, song_id}, {id}
- Khóa chính: {id}

14. Bảng SONG_COMMENTS (**<u>comment_id</u>**, **#user_id**, **#song_id**, comment_text, timestamp_position, created_at)

- Tập phụ thuộc hàm: comment_id → ALL
- Các khóa dự tuyển: {comment_id}
- Khóa chính: {comment_id}

15. Bảng COMMENT_LIKES (**<u>id</u>**, **#user_id**, **#comment_id**, liked_at)

- Tập phụ thuộc hàm: user_id, comment_id → liked_at
- Các khóa dự tuyển: {id}, {user_id, comment_id}
- Khóa chính: {id}

16. Bảng SONG_LIKES (**<u>id</u>**, **#user_id**, **#song_id**, liked_at)

- Tập phụ thuộc hàm: user_id, song_id → liked_at
- Các khóa dự tuyển:{id}, {user_id, song_id}
- Khóa chính: {user_id, song_id}

17. Bảng USER_SONGS (**<u>listen_id</u>**, **#user_id**, **#song_id**, listened_at)

- Tập phụ thuộc hàm: listen_id → ALL
- Các khóa dự tuyển: {listen_id}
- Khóa chính: {listen_id}

Lưu ý: USER_SONGS sử dụng surro gate key (listen_id) vì một user được phép nghe cùng một bài hát nhiều lần tại nhiều thời điểm khác nhau.

C. Chuẩn hóa

1. Bảng USERS (user_id, username, email, password_hash, avatar_url, created_at, #role_id)

Bước 1: Kiểm tra Dạng chuẩn 1 (1NF)

- Định nghĩa: "Mỗi ô chỉ chứa 1 giá trị duy nhất".
- Xét duyệt: Tất cả các thuộc tính (username, email, password_hash…) đều chứa giá trị đơn nguyên tử, không có thuộc tính nào chứa danh sách giá trị.
- Kết luận: Bảng đạt chuẩn 1NF.

Bước 2: Kiểm tra Dạng chuẩn 2 (2NF)

- Định nghĩa: "Các thuộc tính không phải khóa phải phụ thuộc vào toàn bộ khóa chính".
- Xét duyệt: Khóa chính là user_id chỉ bao gồm 1 thuộc tính duy nhất. Theo lý thuyết, phụ thuộc bộ phận chỉ xảy ra khi khóa chính là khóa phức hợp. Vì khóa chính đơn, không tồn tại phụ thuộc bộ phận.
- Kết luận: Bảng đạt chuẩn 2NF.

Bước 3: Kiểm tra Dạng chuẩn 3 (3NF)

- Định nghĩa: "Thuộc tính không khóa chỉ được phụ thuộc vào Khóa chính, không phụ thuộc bắc cầu (A → B → C)."
- Xét duyệt: Các thuộc tính như password_hash, avatar_url, created_at, role_id đều phụ thuộc trực tiếp vào user_id. Không tồn tại phụ thuộc bắc cầu.
- Kết luận: Bảng đạt chuẩn 3NF.

Bước 4: Kiểm tra Dạng chuẩn BCNF

- Định nghĩa: "Mọi yếu tố quyết định đều phải là Siêu khóa".
- Xét duyệt các phụ thuộc hàm:
    1. Xét user_id → ALL: user_id là Khóa chính → là Siêu khóa. (Thỏa mãn)
    2. Xét username → ALL: username là Khóa dự tuyển → là Siêu khóa. (Thỏa mãn)
    3. Xét email → ALL: email là Khóa dự tuyển → là Siêu khóa. (Thỏa mãn)
- Kết luận: Tất cả các vế trái của phụ thuộc hàm đều là siêu khóa → Bảng đạt chuẩn BCNF.

2. Các bảng thực thể đơn (ROLES, SUBSCRIPTION_PLANS, ARTISTS, ALBUMS, SONGS, GENRES, PLAYLISTS, SONG_COMMENTS)

Các bảng này có đặc điểm chung là sử dụng **Khóa chính đơn (Single Key)** (chỉ gồm 1 cột ID).

Kiểm tra:

- **Đạt 1NF**: Các thuộc tính đều mang giá trị đơn nguyên tử.
- **Đạt 2NF**: Vì khóa chính chỉ có 1 thuộc tính (ví dụ: song_id, album_id), không thể xảy ra hiện tượng phụ thuộc bộ phận vào khóa. Mọi thuộc tính đều phụ thuộc vào toàn bộ khóa chính.
- **Đạt 3NF & BCNF**: Các bảng đã được thiết kế tách biệt. Các thông tin liên quan đến đối tượng khác đều được tham chiếu qua Khóa ngoại (Foreign Key) thay vì lưu trực tiếp.

Kết luận: Các bảng ROLES, SUBSCRIPTION_PLANS, ARTISTS, ALBUMS, SONGS, GENRES, PLAYLISTS, SONG_COMMENTS đều đạt chuẩn BCNF.

3. Kiểm tra chuẩn hóa cho nhóm quan hệ có khóa phức hợp (PLAYLIST_SONGS, SONG_ARTISTS, USER_SONGS)

a. Quan hệ PLAYLIST_SONGS

- Lược đồ quan hệ: (**id**, **#playlist_id**, **#song_id**, position, added_at)
- Khóa chính (PK): {id}
- Phụ thuộc hàm (FD): id → playlist_id, song_id, position, added_at

Chứng minh đạt chuẩn 2NF:

- Điều kiện: Một quan hệ đạt 2NF nếu đã đạt 1NF và các thuộc tính không khóa phụ thuộc đầy đủ vào khóa chính, không phụ thuộc bộ phận.
- Xét thuộc tính không khóa: position, added_at.
- Biện luận:
    - playlist_id → added_at: Chỉ biết Playlist thì không xác định được thời điểm thêm của một bài hát cụ thể. (Không thỏa mãn)
    - song_id → added_at: Chỉ biết bài hát thì không xác định được nó được thêm vào Playlist nào và khi nào. (Không thỏa mãn)
    - Phải tồn tại đồng thời cả {playlist_id, song_id} hoặc {id} mới xác định duy nhất position và added_at.
- Kết luận: Đạt chuẩn 2NF.

Chứng minh đạt chuẩn BCNF:

- Phụ thuộc hàm là {id}, {playlist_id, song_id} → position, added_at.
- Vế trái {id} chính là Khóa chính (và là siêu khóa).
- Kết luận: Đạt chuẩn BCNF.

b. Quan hệ SONG_ARTISTS

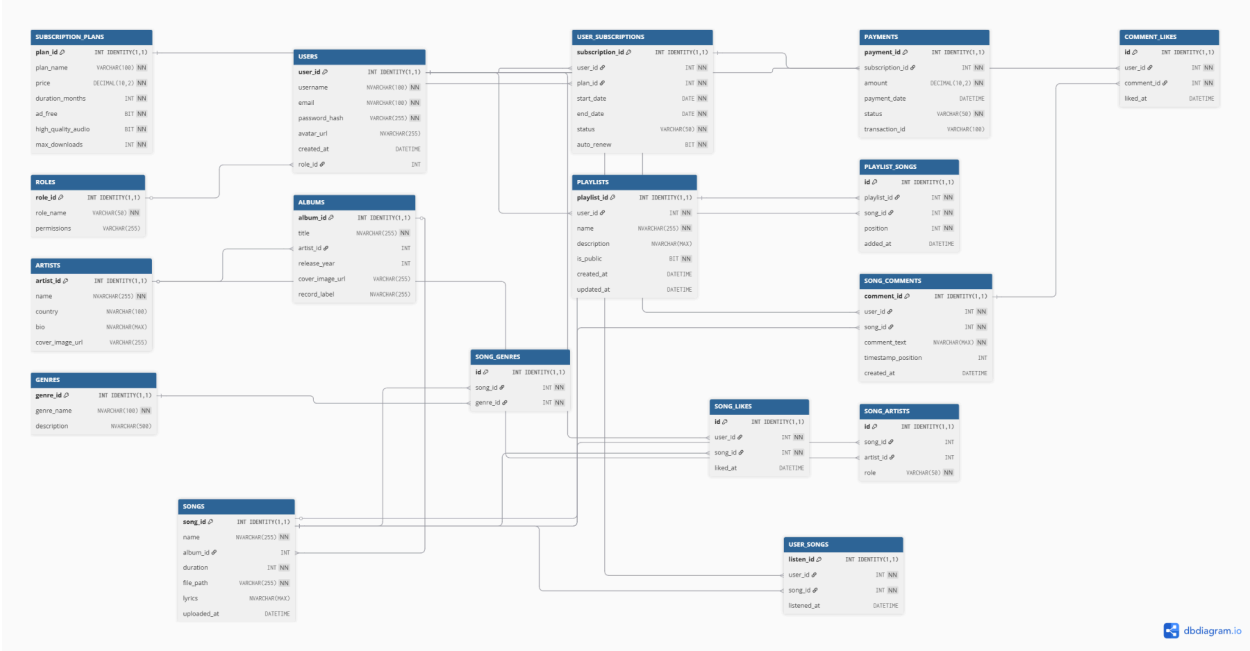
- Lược đồ quan hệ: (**<u>id</u>**, **#song_id**, **#artist_id**, role)
- Khóa chính (PK): {id}
- Phụ thuộc hàm: song_id, artist_id → role, id → role, song_id,artist_id
- Biện luận: Chỉ khi biết cả bài hát lẫn nghệ sĩ mới xác định được vai trò của nghệ sĩ trong bài hát đó.
- Kết luận: Đạt chuẩn BCNF.

c. Quan hệ USER_SONGS (lịch sử nghe)

- Lược đồ: (**<u>listen_id</u>**, **#user_id**, **#song_id**, listened_at)
- Sử dụng surrogate key listen_id vì cần cho phép một user nghe cùng một bài hát nhiều lần.
- Phụ thuộc hàm: listen_id → ALL
- Kết luận: Đạt chuẩn BCNF.

Relational Schema:

https://dbdiagram.io/d/Copy-of-Untitled-Diagram-68d3a3e6d2b621e422ae3f96



V. Phân tích ràng buộc

1. Ràng buộc miền giá trị (Domain Constraints)

a. Ràng buộc duy nhất (UNIQUE)

- **USERS**: username, email (Không cho phép hai người dùng trùng thông tin định danh).
- **SONGS**: file_path (Tránh lưu trữ trùng lặp tệp tin vật lý trên máy chủ).
- **PAYMENTS**: transaction_id (Mã giao dịch phải là duy nhất toàn hệ thống).
- **GENRES**: genre_name (Không trùng tên thể loại).
- **Khóa chính (PK)**: Tất cả các cột mã ID (user_id, song_id, artist_id, album_id, playlist_id, …) phải là duy nhất để định danh thực thể.

b. Ràng buộc bắt buộc (NOT NULL)

- Thông tin định danh và bảo mật: *user_id, username, email, password_hash*.
- Thông tin cốt lõi bài hát: *song_id, name, file_path, duration*.
- Thông tin sở hữu: *playlist_id, user_id* (Mọi Playlist phải có chủ sở hữu rõ ràng).
- Thông tin đăng ký: *user_id, plan_id, start_date, end_date, status*.
- Thông tin thanh toán: *subscription_id, amount, status*.

c. Ràng buộc kiểm tra (CHECK)

Bảng	Cột	Điều kiện	Ý nghĩa
SUBSCRIPTION_PLANS	price	price >= 0	Giá không âm
SUBSCRIPTION_PLANS	duration_months	duration_months > 0	Thời hạn phải dương
SUBSCRIPTION_PLANS	max_downloads	max_downloads >= 0	Số lượt tải không âm
USER_SUBSCRIPTIONS	status	IN ('active','expired','cancelled','paused')	Trạng thái hợp lệ
PAYMENTS	amount	amount > 0	Số tiền phải dương
PAYMENTS	status	IN ('pending','completed','failed','refunded')	Trạng thái thanh toán hợp lệ
SONGS	duration	duration > 0	Thời lượng bài hát phải là số dương
SONG_ARTISTS	role	IN ('main','featured','composer','producer')	Vai trò hợp lệ
PLAYLIST_SONGS	position	position > 0	Vị trí phải là số dương
SONG_COMMENTS	timestamp_position	timestamp_position >= 0	Thời điểm bình luận không âm

d. Giá trị mặc định (DEFAULT)

- **created_at, listened_at, payment_date, uploaded_at, added_at**: GETDATE() — Tự động ghi lại thời điểm tạo.
- **is_public**: 1 (TRUE) — Mặc định các danh sách phát mới tạo sẽ ở chế độ công khai.
- **ad_free, high_quality_audio**: 0 — Mặc định gói Free không có đặc quyền.
- **max_downloads**: 0 — Mặc định không cho phép tải nhạc offline.
- **auto_renew**: 0 — Mặc định không tự động gia hạn.

2. Ràng buộc toàn vẹn tham chiếu (Referential Integrity)

Xác định mối quan hệ giữa các bảng (Foreign Keys) và hành động tương ứng khi dữ liệu gốc bị thay đổi hoặc xóa bỏ.

a. Phân tích các mối quan hệ (Cardinality)

Quan hệ 1-N:

- **ROLES → USERS**: Một vai trò có thể được gán cho nhiều người dùng.

- **USERS → USER_SUBSCRIPTIONS**: Một người dùng có thể đăng ký nhiều gói.

- **SUBSCRIPTION_PLANS → USER_SUBSCRIPTIONS**: Một gói có thể được nhiều người dùng đăng ký.

- **USER_SUBSCRIPTIONS → PAYMENTS**: Một lần đăng ký có thể có nhiều giao dịch thanh toán.

- **ARTISTS → ALBUMS**: Một nghệ sĩ có thể phát hành nhiều album.

- **ALBUMS → SONGS**: Một album chứa nhiều bài hát.

- **USERS → PLAYLISTS**: Một người dùng có thể tạo nhiều danh sách phát.

- **SONGS → SONG_COMMENTS**: Một bài hát có nhiều bình luận.

Quan hệ N-N (Sử dụng bảng trung gian):

- **SONGS ↔ ARTISTS**: Bảng trung gian **SONG_ARTISTS** (thêm thuộc tính role).

- **SONGS ↔ GENRES**: Bảng trung gian **SONG_GENRES**.

- **PLAYLISTS ↔ SONGS**: Bảng trung gian **PLAYLIST_SONGS** (thêm position, added_at).

- **USERS ↔ SONGS**: Bảng trung gian **USER_SONGS** (lịch sử nghe nhạc).

- **USERS ↔ SONG_COMMENTS**: Bảng trung gian **COMMENT_LIKES.**

- **USERS ↔ SONGS**: Bảng trung gian **SONG_LIKES** (lượt thích bài hát).

b. Hành động khi xóa dữ liệu (ON DELETE)

Dựa trên quy tắc nghiệp vụ đã xác định, các hành động xóa được thiết lập như sau để bảo vệ luồng dữ liệu:

Child Table	FK Column	Parent Table	ON DELETE	Ý nghĩa nghiệp vụ
USERS	role_id	**ROLES**	NO ACTION	Không cho xóa vai trò nếu còn user đang dùng — bảo vệ cấu trúc phân quyền
USER_SUBSCRIPTIONS	user_id	**USERS**	CASCADE	Xóa user → hủy toàn bộ gói đăng ký của họ
USER_SUBSCRIPTIONS	plan_id	**SUBSCRIPTION_PLANS**	NO ACTION	Không cho xóa gói nếu còn người đăng ký — bảo vệ dữ liệu kinh doanh
PAYMENTS	subscription_id	**USER_SUBSCRIPTIONS**	CASCADE	Xóa gói đăng ký → xóa các payments liên quan
ALBUMS	artist_id	**ARTISTS**	NO ACTION*	Không cho xóa nghệ sĩ nếu còn album — phải xóa/chuyển album trước
SONGS	album_id	**ALBUMS**	SET NULL	Xóa album → bài hát vẫn tồn tại, album_id = NULL (giống single không album)
SONG_ARTISTS	song_id	**SONGS**	CASCADE	Xóa bài hát → xóa liên kết nghệ sĩ-bài hát
SONG_ARTISTS	artist_id	**ARTISTS**	CASCADE	Xóa nghệ sĩ → gỡ tên họ khỏi bảng trung gian (bài hát vẫn còn)
SONG_GENRES	song_id	**SONGS**	CASCADE	Xóa bài hát → xóa thể loại gắn với bài đó
SONG_GENRES	genre_id	**GENRES**	CASCADE	Xóa thể loại → gỡ tag thể loại khỏi tất cả bài hát
PLAYLISTS	user_id	**USERS**	CASCADE	Xóa user → xóa tất cả playlist họ tạo
PLAYLIST_SONGS	playlist_id	**PLAYLISTS**	CASCADE	Xóa playlist → xóa danh sách bài hát bên trong

Child Table	FK Column	Parent Table	ON DELETE	Ý nghĩa nghiệp vụ
PLAYLIST_SONGS	song_id	**SONGS**	CASCADE	Xóa bài hát → gỡ khỏi tất cả playlist
SONG_COMMENTS	user_id	**USERS**	CASCADE	Xóa user → xóa tất cả bình luận họ viết
SONG_COMMENTS	song_id	**SONGS**	CASCADE	Xóa bài hát → xóa tất cả bình luận trên bài đó
COMMENT_LIKES	user_id	**USERS**	NO ACTION*	Không thể xóa người dùng nếu như họ conf bình luận có like
COMMENT_LIKES	comment_id	**SONG_COMMENTS**	CASCADE	Xóa bình luận → xóa tất cả lượt like của bình luận đó
SONG_LIKES	user_id	**USERS**	CASCADE	Xóa user → xóa tất cả lượt thích bài hát của họ
SONG_LIKES	song_id	**SONGS**	CASCADE	Xóa bài hát → xóa tất cả lượt thích của bài đó
USER_SONGS	user_id	**USERS**	CASCADE	Xóa user → xóa lịch sử nghe nhạc của họ
USER_SONGS	song_id	**SONGS**	CASCADE	Xóa bài hát → xóa lịch sử nghe bài đó

*SQL Server ngăn chặn nghiêm ngặt việc có nhiều luồng ON DELETE CASCADE cùng dẫn đến một bảng, nhằm tránh sự nhập nhằng trong thứ tự thực thi và nguy cơ xảy ra vòng lặp vô hạn. Nếu ta bắt buộc phải xóa hoàn toàn (hard-delete) người dùng và muốn database tự động dọn dẹp dữ liệu liên quan mà không gặp lỗi ON DELETE CASCADE, ta có thể sử dụng Trigger trong SQL Server.

VI. Cài đặt CSDL

1. Cài đặt bảng

```
create database music_streaming;
GO
use music_streaming;
GO
```

Bảng ROLES

```
CREATE TABLE ROLES (
    role_id INT IDENTITY(1,1) PRIMARY KEY,
    role_name VARCHAR(50) NOT NULL, -- admin/user/moderator
    permissions VARCHAR(255)
);
```

Bảng USERS

```
CREATE TABLE USERS (
    user_id INT IDENTITY(1,1) PRIMARY KEY,
    username NVARCHAR(100) UNIQUE NOT NULL,
    email NVARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    avatar_url NVARCHAR(255),
    created_at DATETIME DEFAULT GETDATE(),
    role_id INT FOREIGN KEY REFERENCES ROLES(role_id)
);
```

Bảng SUBSCRIPTION_PLANS

```
CREATE TABLE SUBSCRIPTION_PLANS (
    plan_id INT IDENTITY(1,1) PRIMARY KEY,
    plan_name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL CHECK (price >= 0),
    duration_months INT NOT NULL CHECK (duration_months > 0),
    ad_free BIT NOT NULL DEFAULT 0,
    high_quality_audio BIT NOT NULL DEFAULT 0,
```

```
    max_downloads INT NOT NULL DEFAULT 0 CHECK (max_downloads >= 0)
);
```

Bảng USER_SUBSCRIPTIONS

```
CREATE TABLE USER_SUBSCRIPTIONS (
    subscription_id INT IDENTITY(1,1) PRIMARY KEY,
    user_id INT NOT NULL FOREIGN KEY REFERENCES USERS(user_id) ON
DELETE CASCADE,
    plan_id INT NOT NULL FOREIGN KEY REFERENCES
SUBSCRIPTION_PLANS(plan_id),
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    status VARCHAR(50) NOT NULL CHECK (status IN ('active',
'expired', 'cancelled', 'paused')),
    auto_renew BIT NOT NULL DEFAULT 0
);
```

Bảng PAYMENTS

```
CREATE TABLE PAYMENTS (
    payment_id INT IDENTITY(1,1) PRIMARY KEY,
    subscription_id INT NOT NULL FOREIGN KEY REFERENCES
USER_SUBSCRIPTIONS(subscription_id) ON DELETE CASCADE,
    amount DECIMAL(10,2) NOT NULL CHECK (amount > 0),
    payment_date DATETIME DEFAULT GETDATE(),
    status VARCHAR(50) NOT NULL CHECK (status IN ('pending',
'completed', 'failed', 'refunded')),
    transaction_id VARCHAR(100) UNIQUE
);
```

Bảng ARTISTS

```sql
CREATE TABLE ARTISTS (
    artist_id INT IDENTITY(1,1) PRIMARY KEY,
    name NVARCHAR(255) NOT NULL,
    country NVARCHAR(100),
    bio NVARCHAR(MAX),
    cover_image_url VARCHAR(255)
);
```

Bảng ALBUMS

```sql
CREATE TABLE ALBUMS (
    album_id INT IDENTITY(1,1) PRIMARY KEY,
    title NVARCHAR(255) NOT NULL,
    artist_id INT FOREIGN KEY REFERENCES ARTISTS(artist_id),
    release_year INT,
    cover_image_url VARCHAR(255),
    record_label NVARCHAR(255)
);
```

Bảng SONGS

```sql
CREATE TABLE SONGS (
    song_id INT IDENTITY(1,1) PRIMARY KEY,
    name NVARCHAR(255) NOT NULL,
    album_id INT FOREIGN KEY REFERENCES ALBUMS(album_id) ON DELETE
SET NULL,
    duration INT NOT NULL CHECK (duration > 0),
    file_path VARCHAR(255) UNIQUE NOT NULL,
    lyrics NVARCHAR(MAX),
    uploaded_at DATETIME DEFAULT GETDATE()
);
```

Bảng SONG_ARTISTS (N-N)

```sql
CREATE TABLE SONG_ARTISTS (
    id INT IDENTITY(1,1) PRIMARY KEY,
```

```
    song_id INT FOREIGN KEY REFERENCES SONGS(song_id) ON DELETE
CASCADE,
    artist_id INT FOREIGN KEY REFERENCES ARTISTS(artist_id) ON DELETE
CASCADE,
    role VARCHAR(50) NOT NULL
);
```

Bảng GENRES

```
CREATE TABLE GENRES (
    genre_id INT IDENTITY(1,1) PRIMARY KEY,
    genre_name NVARCHAR(100) UNIQUE NOT NULL,
    description NVARCHAR(500)
);
```

Bảng SONG_GENRES (N-N)

```
CREATE TABLE SONG_GENRES (
    id INT IDENTITY(1,1) PRIMARY KEY,
    song_id INT NOT NULL FOREIGN KEY REFERENCES SONGS(song_id) ON
DELETE CASCADE,
    genre_id INT NOT NULL FOREIGN KEY REFERENCES GENRES(genre_id) ON
DELETE CASCADE,
    UNIQUE (song_id, genre_id)
);
```

Bảng PLAYLISTS

```
CREATE TABLE PLAYLISTS (
    playlist_id INT IDENTITY(1,1) PRIMARY KEY,
    user_id INT NOT NULL FOREIGN KEY REFERENCES USERS(user_id) ON
DELETE CASCADE,
    name NVARCHAR(255) NOT NULL,
    description NVARCHAR(MAX),
    is_public BIT NOT NULL DEFAULT 1,
    created_at DATETIME DEFAULT GETDATE(),
    updated_at DATETIME DEFAULT GETDATE()
```

```
);
```

Bảng PLAYLIST_SONGS (N-N)

```
CREATE TABLE PLAYLIST_SONGS (
    id INT IDENTITY(1,1) PRIMARY KEY,
    playlist_id INT NOT NULL FOREIGN KEY REFERENCES
PLAYLISTS(playlist_id) ON DELETE CASCADE,
    song_id INT NOT NULL FOREIGN KEY REFERENCES SONGS(song_id) ON
DELETE CASCADE,
    position INT NOT NULL CHECK (position > 0),
    added_at DATETIME DEFAULT GETDATE(),
    UNIQUE (playlist_id, song_id)
);
```

Bảng SONG_COMMENTS

```
CREATE TABLE SONG_COMMENTS (
    comment_id INT IDENTITY(1,1) PRIMARY KEY,
    user_id INT NOT NULL FOREIGN KEY REFERENCES USERS(user_id) ON
DELETE CASCADE,
    song_id INT NOT NULL FOREIGN KEY REFERENCES SONGS(song_id) ON
DELETE CASCADE,
    comment_text NVARCHAR(MAX) NOT NULL,
    timestamp_position INT CHECK (timestamp_position >= 0),
    created_at DATETIME DEFAULT GETDATE()
);
```

Bảng COMMENT_LIKES

```
CREATE TABLE COMMENT_LIKES (
    id INT IDENTITY(1,1) PRIMARY KEY,
    user_id INT NOT NULL FOREIGN KEY REFERENCES USERS(user_id) ON
DELETE CASCADE,
    comment_id INT NOT NULL FOREIGN KEY REFERENCES
SONG_COMMENTS(comment_id) ON DELETE NO ACTION,
    liked_at DATETIME DEFAULT GETDATE(),
    UNIQUE (user_id, comment_id)
);
```

Bảng SONG_LIKES

```
CREATE TABLE SONG_LIKES (
    id INT IDENTITY(1,1) PRIMARY KEY,
    user_id INT NOT NULL FOREIGN KEY REFERENCES USERS(user_id) ON
DELETE CASCADE,
    song_id INT NOT NULL FOREIGN KEY REFERENCES SONGS(song_id)ON
DELETE CASCADE,
    liked_at DATETIME DEFAULT GETDATE(),
    UNIQUE (user_id, song_id)
);
```

Bảng USER_SONGS (lịch sử nghe)

```
CREATE TABLE USER_SONGS (
    listen_id INT IDENTITY(1,1) PRIMARY KEY,
    user_id INT NOT NULL FOREIGN KEY REFERENCES USERS(user_id) ON
DELETE CASCADE,
    song_id INT NOT NULL FOREIGN KEY REFERENCES SONGS(song_id) ON
DELETE CASCADE,
    listened_at DATETIME DEFAULT GETDATE()
);
GO
```

2. Dữ liệu mẫu được nhập vào

a) Bảng ROLES

INSERT INTO ROLES (role_name, permissions) **VALUES**
('admin', 'ALL'),
('user', 'BASIC'),
('moderator', 'MODERATE');

b) Bảng USERS

INSERT INTO USERS (username, email, password_hash, avatar_url, role_id) **VALUES**
('admin1', 'admin1@mail.com', 'hash1', **NULL**, 1),
('mod1', 'mod1@mail.com', 'hash2', **NULL**, 3),
('user1', 'user1@mail.com', 'hash3', **NULL**, 2),
('user2', 'user2@mail.com', 'hash4', **NULL**, 2),

('user3', 'user3@mail.com', 'hash5', **NULL**, 2);
-- ... *(20 bản ghi tổng: 1 admin, 1 moderator, 18 users)*

c) Bảng SUBSCRIPTION_PLANS

INSERT INTO SUBSCRIPTION_PLANS
(plan_name, price, duration_months, ad_free, high_quality_audio, max_downloads)
VALUES
('Free', 0, 1, 0, 0, 0),
('Premium', 9.99, 1, 1, 1, 100),
('Family', 19.99, 1, 1, 1, 500);

d) Bảng ARTISTS

INSERT INTO ARTISTS (name, country, bio, cover_image_url) **VALUES**
(N'Sơn Tùng M-TP', N'Vietnam', N'Pop singer', **NULL**),
(N'Jack', N'Vietnam', N'Vpop artist', **NULL**),
(N'BTS', N'Korea', N'Kpop group', **NULL**),
(N'Taylor Swift', N'USA', N'Pop star', **NULL**),
(N'Ed Sheeran', N'UK', N'Pop artist', **NULL**);
-- ... *(20 nghệ sĩ tổng)*

e) Bảng ALBUMS

INSERT INTO ALBUMS (title, artist_id, release_year, cover_image_url, record_label) **VALUES**
(N'Album 1', 1, 2020, **NULL**, N'MTP'),
(N'Album 2', 2, 2021, **NULL**, N'J97'),
(N'Album 3', 3, 2022, **NULL**, N'Hybe'),
(N'Album 4', 4, 2023, **NULL**, N'Republic'),
(N'Album 5', 5, 2020, **NULL**, N'Atlantic');
-- ... *(20 album tổng)*

f) Bảng SONGS

INSERT INTO SONGS (name, album_id, duration, file_path, lyrics) **VALUES**
(N'Song 1', 1, 210, '/music/song1.mp3', **NULL**),
(N'Song 2', 2, 200, '/music/song2.mp3', **NULL**),
(N'Song 3', 3, 180, '/music/song3.mp3', **NULL**);
-- ... *(25 bài hát tổng)*

g) Bảng SONG_ARTISTS

INSERT INTO SONG_ARTISTS (song_id, artist_id, **role**) **VALUES**
(1, 1, 'main'),
(2, 2, 'main'),
-- ... *bài 21-25 bổ sung nghệ sĩ featured*
(21, 1, 'featured'),
(22, 2, 'featured');

h) Bảng GENRES

INSERT INTO GENRES (genre_name, description) **VALUES**
(N'Pop', N'Popular music'),

```
(N'Rock',   N'Rock music'),
(N'Hip Hop',N'Hip Hop music'),
(N'Kpop',   N'Korean pop'),
(N'Vpop',   N'Vietnamese pop');
-- ... (20 thể loại tổng)
```

i) Bảng USER_SUBSCRIPTIONS

```
INSERT INTO USER_SUBSCRIPTIONS
    (user_id, plan_id, start_date, end_date, status, auto_renew) VALUES
(1, 2, '2025-01-01', '2025-02-01', 'active',  1),
(2, 2, '2025-01-05', '2025-02-05', 'active',  1),
(3, 1, '2025-01-01', '2025-02-01', 'active',  0);
-- ... (40 bản ghi tổng — mỗi user đăng ký 2 lần)
```

j) Bảng PLAYLISTS

```
INSERT INTO PLAYLISTS (user_id, name, description, is_public) VALUES
(1, N'Admin Playlist', N'Admin favorites', 1),
(2, N'Mod Playlist',   N'Moderator picks', 1),
(3, N'User1 Playlist', N'My songs',        1);
-- ... (20 playlist tổng)
```

k) Bảng USER_SONGS (lịch sử nghe — dữ liệu ngẫu nhiên)

```
INSERT INTO USER_SONGS (user_id, song_id, listened_at)
SELECT TOP 500
    u.user_id,
    s.song_id,
    DATEADD(DAY, -ABS(CHECKSUM(NEWID()) % 14), GETDATE())
FROM USERS u
CROSS JOIN SONGS s
ORDER BY NEWID();
```

VII. Thử nghiệm với CSDL đã thiết kế

Câu hỏi 1: Liệt kê tất cả users

```
SELECT * FROM Users;
```

```
1  SELECT * FROM Users;
```

214 % 1 0

Results | Messages

	user_id	username	email	password_hash	avatar_url	created_at	role_id
1	1	user_1	user1@mail.com	hash_abc123	https://avatar.com/user1.jpg	2026-03-18 01:55:05.247	2
2	2	user_2	user2@mail.com	hash_def456	https://avatar.com/user2.jpg	2026-03-18 01:55:05.247	2
3	3	user_3	user3@mail.com	hash_ghi789	https://avatar.com/user3.jpg	2026-03-18 01:55:05.247	2
4	4	user_4	user4@mail.com	hash_jkl012	https://avatar.com/user4.jpg	2026-03-18 01:55:05.247	2
5	5	user_5	user5@mail.com	hash_mno345	https://avatar.com/user5.jpg	2026-03-18 01:55:05.247	2
6	6	admin_1	admin@mail.com	hash_pqr678	https://avatar.com/admin.jpg	2026-03-18 01:55:05.247	1
7	7	mod_1	mod@mail.com	hash_stu901	https://avatar.com/mod.jpg	2026-03-18 01:55:05.247	3
8	8	user_6	user6@mail.com	hash_vwx234	https://avatar.com/user6.jpg	2026-03-18 01:55:05.247	2

Câu hỏi 2: Tìm bài hát theo tên

$$\sigma_{name \text{ LIKE } '\%N\acute{a}ng\%'}(SONGS)$$

```
SELECT * FROM Songs
WHERE name LIKE N'%Nắng%';
```

```
1  SELECT * FROM Songs
2  WHERE name LIKE N'%Nắng%';
```

214 % 1 0

Results | Messages

	song_id	name	album_id	duration	file_path	lyrics	uploaded_at
1	6	Nắng Ấm Xa Dần	1	250	/music/nang_am_xa_dan.mp3	Nắng ấm nay đã xa dần...	2026-03-18 01:55:05.337
2	23	Nắng Có Mang Em Về	3	290	/music/nang_co_mang.mp3	Nắng có mang em về nơi đó...	2026-03-18 01:55:05.337

Câu hỏi 3: Danh sách nghệ sĩ Việt Nam

$$\sigma_{country='Vi\hat{e}tNam'}(ARTISTS)$$

```
SELECT * FROM Artists
WHERE Country = N'Việt Nam';
```

```sql
SELECT * FROM Artists
WHERE Country = N'Việt Nam';
```

161 % 1 0

Results Messages

	artist_id	name	country	bio	cover_image_url
1	1	Sơn Tùng M-TP	Việt Nam	Ca sĩ, nhạc sĩ nổi tiếng Việt Nam	https://img.com/sontung.jpg
2	2	Đen Vâu	Việt Nam	Rapper hàng đầu Việt Nam	https://img.com/denvau.jpg
3	3	Hòa Minzy	Việt Nam	Ca sĩ nhạc Pop Việt Nam	https://img.com/hoaminzy.jpg
4	5	Trúc Nhân	Việt Nam	Ca sĩ sáng tạo Việt Nam	https://img.com/trucnhan.jpg
5	6	Bích Phương	Việt Nam	Ca sĩ nhạc Pop Việt Nam	https://img.com/bichphuong.jpg
6	7	Jack	Việt Nam	Ca sĩ trẻ tài năng	https://img.com/jack.jpg
7	10	Karik	Việt Nam	Rapper nổi tiếng Việt Nam	https://img.com/karik.jpg
8	11	Mỹ Tâm	Việt Nam	Họa mi tóc nâu, diva nhạc Việt	https://img.com/mytam.jpg
9	12	Đông Nhi	Việt Nam	Ca sĩ nhạc Pop năng động	https://img.com/dongnhi.jpg
10	13	Noo Phước Thịnh	Việt Nam	Ca sĩ, vũ công nổi tiếng	https://img.com/noophuocthinh.jpg
11	14	MIN	Việt Nam	Ca sĩ trẻ phong cách hiện đại	https://img.com/min.jpg
12	15	Vũ.	Việt Nam	Ca sĩ Indie nổi bật	https://img.com/vu.jpg
13	19	Phan Mạnh Quỳnh	Việt Nam	Nhạc sĩ tài năng, ca sĩ Việt Nam	https://img.com/phanmanhquynh.jpg
14	20	Tóc Tiên	Việt Nam	Ca sĩ phong cách hiện đại, quyến rũ	https://img.com/toctien.jpg

Câu hỏi 4: Top 10 bài hát dài nhất

$$\sigma_{is_public=1}(PLAYLISTS)$$

```sql
SELECT TOP 10 * FROM Songs
ORDER BY Duration DESC;
```

Câu hỏi 5: Tìm các Playlist công khai

```sql
SELECT * FROM Playlists
WHERE is_public = 1;
```

```sql
SELECT * FROM Playlists
WHERE is_public = 1;
```

161 % 1 0

Results | Messages

	playlist_id	user_id	name	description	is_public	created_at	updated_at
1	1	1	Nhạc Chill Cuối Tuần	Danh sách nhạc thư giãn cuối tuần	1	2026-01-15 10:00:00.000	2026-03-18 01:55:05.403
2	2	2	Vpop Hay Nhất 2026	Tổng hợp nhạc Vpop hay nhất	1	2026-02-01 11:00:00.000	2026-03-18 01:55:05.403
3	3	4	Nhạc Của Tui	Playlist cá nhân yêu thích	1	2026-02-10 12:00:00.000	2026-03-18 01:55:05.403
4	5	5	Party Mix	Nhạc sôi động cho party	1	2026-03-01 14:00:00.000	2026-03-18 01:55:05.403
5	6	3	Morning Vibes	Nhạc nhẹ nhàng buổi sáng	1	2026-03-05 15:00:00.000	2026-03-18 01:55:05.403
6	7	11	Tình Khúc Bất Hủ	Những bài hát tình cảm kinh điển	1	2026-01-20 08:00:00.000	2026-03-18 01:55:05.403
7	8	12	Workout Playlist	Nhạc tập gym sôi động	1	2026-02-05 09:00:00.000	2026-03-18 01:55:05.403
8	9	13	Indie Việt	Nhạc Indie Việt Nam hay nhất	1	2026-02-12 10:00:00.000	2026-03-18 01:55:05.403
9	10	14	Kpop Favorites	Tuyển tập Kpop yêu thích	1	2026-02-20 11:00:00.000	2026-03-18 01:55:05.403
10	11	15	Study Time	Nhạc nghe khi học bài	1	2026-03-01 12:00:00.000	2026-03-18 01:55:05.403
11	12	16	Road Trip	Nhạc cho chuyến đi xa	1	2026-03-03 13:00:00.000	2026-03-18 01:55:05.403
12	13	17	Ballad Việt	Tuyển tập Ballad hay nhất	1	2026-03-06 14:00:00.000	2026-03-18 01:55:05.403
13	14	18	US-UK Hits	Nhạc US-UK hay nhất 2026	1	2026-03-08 15:00:00.000	2026-03-18 01:55:05.403
14	15	19	Top Rap Việt	Nhạc Rap Việt Nam hay nhất	1	2026-03-10 16:00:00.000	2026-03-18 01:55:05.403
15	16	20	Nhạc Trẻ Hot	Nhạc trẻ thịnh hành nhất	1	2026-03-12 17:00:00.000	2026-03-18 01:55:05.403
16	18	8	EDM Night	Nhạc EDM cho đêm dài	1	2026-02-18 19:00:00.000	2026-03-18 01:55:05.403
17	20	10	Throwback Hits	Nhạc cũ nghe lại vẫn hay	1	2026-03-05 21:00:00.000	2026-03-18 01:55:05.403

Câu hỏi 6: Bài hát của nghệ sĩ X

$$\pi_{SONGS.name,ARTISTS.name}(\sigma_{ARTISTS.name='SonTungM-TP'}(SONGS \bowtie SONG_ARTISTS \bowtie ARTISTS))$$

```sql
SELECT S.name AS SongName, A.name AS ArtistName
FROM SONGS S
JOIN SONG_ARTISTS SA ON S.song_id = SA.song_id
JOIN ARTISTS A ON SA.artist_id = A.artist_id
WHERE A.name = N'Sơn Tùng M-TP';
```

```sql
SELECT S.name AS SongName, A.name AS ArtistName
FROM SONGS S
JOIN SONG_ARTISTS SA ON S.song_id = SA.song_id
JOIN ARTISTS A ON SA.artist_id = A.artist_id
WHERE A.name = N'Sơn Tùng M-TP';
```

161 % 1 0

Results | Messages

	SongName	ArtistName
1	Lạc Trôi	Sơn Tùng M-TP
2	Nơi Này Có Anh	Sơn Tùng M-TP
3	Chạy Ngay Đi	Sơn Tùng M-TP
4	Hãy Trao Cho Anh	Sơn Tùng M-TP
5	Muộn Rồi Mà Sao Còn	Sơn Tùng M-TP
6	Nắng Ấm Xa Dần	Sơn Tùng M-TP

Câu hỏi 7: Bài hát trong album Y

$$\pi_{song_id,name,duration,file_path}(\sigma_{album_id=2}(SONGS))$$

```
SELECT song_id, name, duration, file_path
FROM SONGS
WHERE album_id = 2;
```

```
1  SELECT song_id, name, duration, file_path
2  FROM SONGS
3  WHERE album_id = 2;
```

161 % 1 0

Results | Messages

	song_id	name	duration	file_path
1	3	Chạy Ngay Đi	265	/music/chay_ngay_di.mp3
2	4	Hãy Trao Cho Anh	310	/music/hay_trao_cho_anh.mp3
3	5	Muộn Rồi Mà Sao Còn	295	/music/muon_roi.mp3
4	24	Waiting For You	220	/music/waiting_for_you.mp3

Câu hỏi 8: Playlist của user Z

$$\pi_{playlist_id,name,description,is_public,created_at}(\sigma_{user_id=4}(PLAYLISTS))$$

```
SELECT playlist_id, name, description, is_public, created_at
FROM PLAYLISTS
WHERE user_id = 4;
```

```
1  SELECT playlist_id, name, description, is_public, created_at
2  FROM PLAYLISTS
3  WHERE user_id = 4;
```

161 % 1 0

Results | Messages

	playlist_id	name	description	is_public	created_at
1	3	Nhạc Của Tui	Playlist cá nhân yêu thích	1	2026-02-10 12:00:00.000
2	4	Nhạc Buồn Đêm Khuya	Những bài hát buồn nghe đêm khuya	0	2026-02-15 13:00:00.000

Câu hỏi 9: Nghệ sĩ featured trong bài hát A

$$\pi_{artist_id,name,country}(\sigma_{song_id=15 \wedge role='featured'}(ARTISTS \bowtie SONG_ARTISTS))$$

```sql
SELECT A.artist_id, A.name, A.country
FROM ARTISTS A
JOIN SONG_ARTISTS SA ON A.artist_id = SA.artist_id
WHERE SA.song_id = 15 AND SA.role = 'featured';
```

```sql
1  SELECT A.artist_id, A.name, A.country
2  FROM ARTISTS A
3  JOIN SONG_ARTISTS SA ON A.artist_id = SA.artist_id
4  WHERE SA.song_id = 15 AND SA.role = 'featured';
```

161 % 1 0

Results | Messages

	artist_id	name	country
1	5	Trúc Nhân	Việt Nam

Câu hỏi 10 : Bài hát thuộc thể loại Pop

$$\pi_{song_id,name,duration}(\sigma_{genre_name='Pop'}(SONGS \bowtie SONG_GENRES \bowtie GENRES))$$

```sql
SELECT S.song_id, S.name, S.duration
FROM SONGS S
JOIN SONG_GENRES SG ON S.song_id = SG.song_id
JOIN GENRES G ON SG.genre_id = G.genre_id
WHERE G.genre_name = N'Pop';
```

```sql
1  SELECT S.song_id, S.name, S.duration
2  FROM SONGS S
3  JOIN SONG_GENRES SG ON S.song_id = SG.song_id
4  JOIN GENRES G ON SG.genre_id = G.genre_id
5  WHERE G.genre_name = N'Pop';
```

161 % 1 0

Results | Messages

	song_id	name	duration
1	1	Lạc Trôi	280
2	4	Hãy Trao Cho Anh	310
3	6	Nắng Ấm Xa Dần	250
4	9	How You Like That	190
5	10	Lovesick Girls	210
6	11	Đi Đu Đưa Đi	205

Câu hỏi 11: Comments của bài hát X

$$\tau_{created_at\ \mathrm{DESC}}(\pi_{comment_id,user_id,comment_text,created_at}(\sigma_{song_id=20}(SONG_COMMENTS)))$$

```sql
SELECT comment_id, user_id, comment_text, created_at
FROM SONG_COMMENTS
WHERE song_id = 20
ORDER BY created_at DESC;
```

```sql
1  SELECT comment_id, user_id, comment_text, created_at
2  FROM SONG_COMMENTS
3  WHERE song_id = 20
4  ORDER BY created_at DESC;
```

161 % 1 0

Results Messages

	comment_id	user_id	comment_text	created_at
1	100	5	Comment này sẽ bị xóa bởi admin hoặc chủ sở hữu	2026-03-15 20:00:00.000
2	10	5	Love this track so much!	2026-03-13 11:00:00.000
3	9	3	Can't stop dancing to this	2026-03-12 10:00:00.000
4	8	2	Great song! Can't stop listening	2026-03-11 09:00:00.000
5	7	1	Taylor Swift is the best!	2026-03-10 08:00:00.000

Câu hỏi 12: Top comments được like nhiều nhất

```sql
SELECT TOP 10
    c.comment_id,
    c.user_id,
    c.comment_text,
    COUNT(cl.user_id) AS likes_count
FROM SONG_COMMENTS c
LEFT JOIN COMMENT_LIKES cl ON c.comment_id = cl.comment_id
WHERE c.song_id = 6
GROUP BY  c.comment_id,  c.user_id, c.comment_text
ORDER BY  likes_count DESC;
```

```sql
SELECT TOP 10
    c.comment_id,
    c.user_id,
    c.comment_text,
    COUNT(cl.user_id) AS likes_count
FROM SONG_COMMENTS c
LEFT JOIN COMMENT_LIKES cl ON c.comment_id = cl.comment_id
WHERE c.song_id = 6
GROUP BY  c.comment_id,  c.user_id, c.comment_text
ORDER BY  likes_count DESC;
```

161 % 1 0

Results Messages

	comment_id	user_id	comment_text	likes_count
1	1	1	Bài hát hay quá! Nghe hoài không chán	4
2	2	2	Giai điệu tuyệt vời, rất cảm xúc	3
3	3	3	Nghe đi nghe lại vẫn thấy hay	2
4	5	5	Sơn Tùng hát hay lắm!	2
5	4	4	Nắng ấm xa dần, lòng buồn man mác	1
6	6	1	Replay lần thứ 100 rồi nè	1

Câu hỏi 13: User A đã comment bài nào?

$$\pi_{song_id,name,created_at}(\sigma_{username='user_1'}(SONG_COMMENTS \bowtie SONGS \bowtie USERS))$$

```sql
SELECT DISTINCT S.song_id, S.name, SC.created_at
FROM SONG_COMMENTS SC
JOIN SONGS S ON SC.song_id = S.song_id
JOIN USERS U ON SC.user_id = U.user_id
WHERE U.username = 'user_1';
```

```sql
SELECT DISTINCT S.song_id, S.name, SC.created_at
FROM SONG_COMMENTS SC
JOIN SONGS S ON SC.song_id = S.song_id
JOIN USERS U ON SC.user_id = U.user_id
WHERE U.username = 'user_1';
```

161 % 1 0

Results Messages

	song_id	name	created_at
1	6	Nắng Ấm Xa Dần	2026-03-01 10:00:00.000
2	1	Lạc Trôi	2026-03-05 10:00:00.000
3	3	Chạy Ngay Đi	2026-03-06 11:00:00.000
4	6	Nắng Ấm Xa Dần	2026-03-06 16:00:00.000
5	15	Rời Bỏ	2026-03-07 12:00:00.000
6	33	Cứ Chill Thôi	2026-03-09 17:00:00.000

Câu hỏi 14: Top 5 users nhận được nhiều like nhất từ comment trong bài hát X

```sql
SELECT TOP 5 U.user_id, U.username, COUNT(CL.comment_id) AS
total_likes_received
FROM COMMENT_LIKES CL
JOIN SONG_COMMENTS SC ON CL.comment_id = SC.comment_id
JOIN USERS U ON SC.user_id = U.user_id
WHERE SC.song_id = @song_id
GROUP BY U.user_id, U.username
ORDER BY total_likes_received DESC;
```

```sql
FROM COMMENT_LIKES CL
JOIN SONG_COMMENTS SC ON CL.comment_id = SC.comment_id
JOIN USERS U ON SC.user_id = U.user_id
WHERE SC.song_id = 6
GROUP BY U.user_id, U.username
ORDER BY total_likes_received DESC;
```

161 % 1 0

Results Messages

	user_id	username	total_likes_received
1	1	user_1	5
2	2	user_2	3
3	3	user_3	2
4	5	user_5	2
5	4	user_4	1

Câu hỏi 15: Bài hát có nhiều comments nhất:

```sql
SELECT TOP 1
S.song_id,
S.name AS song_name,
COUNT(SC.comment_id) AS total_comments
FROM SONG_COMMENTS SC
JOIN SONGS S ON SC.song_id = S.song_id
GROUP BY S.song_id, S.name
ORDER BY total_comments DESC;
```

```sql
SELECT TOP 1
S.song_id,
S.name AS song_name,
COUNT(SC.comment_id) AS total_comments
FROM SONG_COMMENTS SC
JOIN SONGS S ON SC.song_id = S.song_id
GROUP BY S.song_id, S.name
ORDER BY total_comments DESC;
```

161 % 1 0

Results | Messages

	song_id	song_name	total_comments
1	6	Nắng Ấm Xa Dần	6

Câu hỏi 16: Comment tại timestamp Y tại bài hát X:

```sql
SELECT
    SC.comment_id,
    U.username,
    SC.comment_text,
    SC.created_at,
    SC.timestamp_position
FROM SONG_COMMENTS SC
JOIN USERS U ON SC.user_id = U.user_id
WHERE SC.song_id = 6
  AND SC.timestamp_position = 30
ORDER BY SC.created_at DESC;
```

```sql
SELECT
    SC.comment_id,
    U.username,
    SC.comment_text,
    SC.created_at,
    SC.timestamp_position
FROM SONG_COMMENTS SC
JOIN USERS U ON SC.user_id = U.user_id
WHERE SC.song_id = 6
```

161 % No issues found

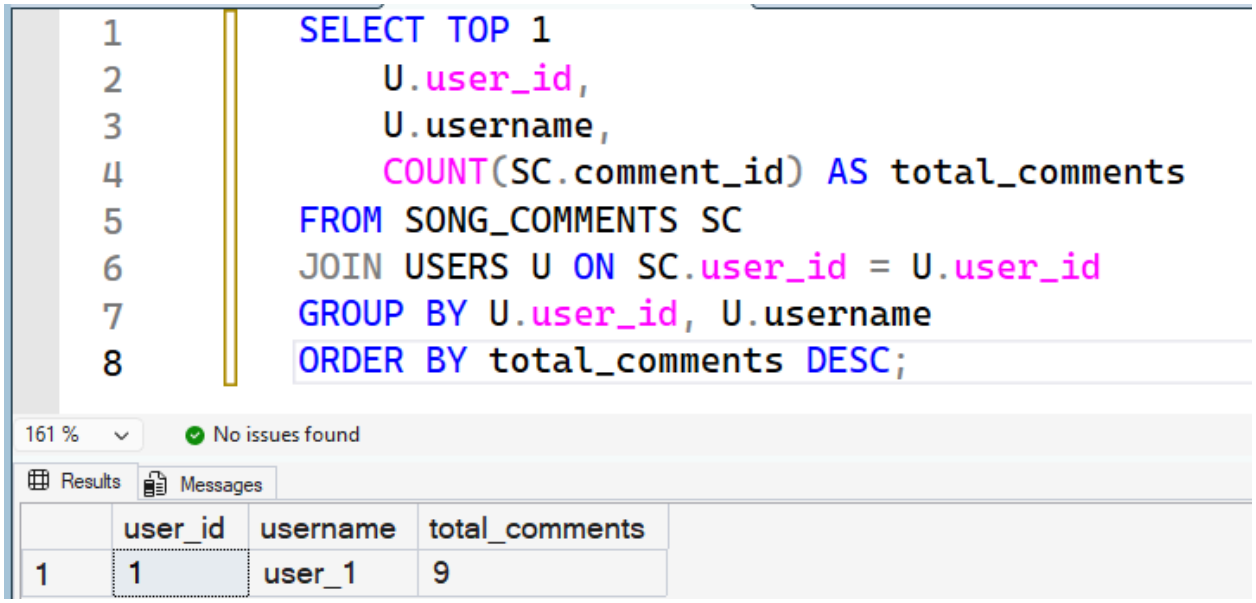
Results | Messages

	comment_id	username	comment_text	created_at	timestamp_position
1	1	user_1	Bài hát hay quá! Nghe hoài không chán	2026-03-01 10:00:00.000	30

Câu hỏi 17: User comment nhiều nhất

```sql
SELECT TOP 1
```

```sql
    U.user_id,
    U.username,
    COUNT(SC.comment_id) AS total_comments
FROM SONG_COMMENTS SC
JOIN USERS U ON SC.user_id = U.user_id
GROUP BY U.user_id, U.username
ORDER BY total_comments DESC;
```



Câu hỏi 18: Ai đã like comment X? (VD: comment_id = 1)

```sql
DECLARE @comment_id INT = 303181;
SELECT
    U.user_id,
    U.username,
    CL.liked_at
FROM COMMENT_LIKES CL
JOIN USERS U ON CL.user_id = U.user_id
WHERE CL.comment_id = @comment_id
ORDER BY CL.liked_at DESC;
```

```sql
DECLARE @comment_id INT = 303181;
SELECT
    U.user_id,
    U.username,
    CL.liked_at
FROM COMMENT_LIKES CL
JOIN USERS U ON CL.user_id = U.user_id
WHERE CL.comment_id = 100
ORDER BY CL.liked_at DESC;
```

161 % No issues found

Results Messages

	user_id	username	liked_at
1	1	user_1	2026-03-16 14:00:00.000
2	4	user_4	2026-03-16 13:00:00.000
3	3	user_3	2026-03-16 12:00:00.000

Câu hỏi 19: Tất cả bài hát của nghệ sĩ X (kể cả featured)

$$\pi_{SONGS.name}(\sigma_{ARTISTS.name='SonTungM-TP'}(SONGS \bowtie SONG_ARTISTS \bowtie ARTISTS))$$

```sql
select s.name as SongName
from dbo.SONGS s
join dbo.SONG_ARTISTS sa on s.song_id = sa.song_id
join dbo.ARTISTS a on sa.artist_id = a.artist_id
where a.name = N'Sơn Tùng M-TP';
```

```sql
select s.name as SongName
from dbo.SONGS s
join dbo.SONG_ARTISTS sa on s.song_id = sa.song_id
join dbo.ARTISTS a on sa.artist_id = a.artist_id
where a.name = N'Sơn Tùng M-TP';
```

161 % No issues found

Results Messages

	SongName
1	Lạc Trôi
2	Nơi Này Có Anh
3	Chạy Ngay Đi
4	Hãy Trao Cho Anh
5	Muộn Rồi Mà Sao Còn
6	Nắng Ấm Xa Dần
7	Waiting For You

Câu hỏi 20: Liệt kê các bài hát có collaborations thuộc thể loại nhạc Vpop

```
select s.name as SongName
from dbo.SONGS s
join dbo.SONG_GENRES sg on s.song_id = sg.song_id
join dbo.GENRES g on sg.genre_id = g.genre_id
where g.genre_name = N'Vpop'
and s.song_id in (
      select song_id
      from SONG_ARTISTS
      group by song_id
      having count(artist_id) >= 2
);
```

```
1   select s.name as SongName
2   from dbo.SONGS s
3   join dbo.SONG_GENRES sg on s.song_id = sg.song_id
4   join dbo.GENRES g on sg.genre_id = g.genre_id
5   where g.genre_name = N'Vpop'
6   and s.song_id in (
7       select song_id
8       from SONG_ARTISTS
9       group by song_id
10      having count(artist_id) >= 2
11  );
```

161 % No issues found

Results Messages

	SongName
1	Hãy Trao Cho Anh
2	Rời Bỏ
3	Sóng Gió
4	Nắng Có Mang Em Về
5	Waiting For You
6	Cô Thắm Không Về
7	Cứ Chill Thôi
8	Em Đây Chẳng Phải Thúy Kiều

Câu hỏi 21: Nghệ sĩ collaboration nhiều nhất

```
select top 1
      a.name as ArtistName,
      count(sa.song_id) as TotalCollabs
from dbo.ARTISTS a
join dbo.SONG_ARTISTS sa on a.artist_id = sa.artist_id
where sa.song_id in (
      select song_id
      from dbo.SONG_ARTISTS
```

```
        group by song_id
        having count(artist_id) >= 2
)
group by a.artist_id, a.name
order by TotalCollabs desc;
```

```
select top 1
    a.name as ArtistName,
    count(sa.song_id) as TotalCollabs
from dbo.ARTISTS a
join dbo.SONG_ARTISTS sa on a.artist_id = sa.artist_id
where sa.song_id in (
    select song_id
    from dbo.SONG_ARTISTS
    group by song_id
    having count(artist_id) >= 2
)
group by a.artist_id, a.name
order by TotalCollabs desc;
```

161 % No issues found

Results Messages

	ArtistName	TotalCollabs
1	Karik	4

Câu hỏi 22: Users đang dùng Premium

```
select
u.user_id,
    u.username,
    u.email,
    us.end_date
from dbo.USERS u
join dbo.USER_SUBSCRIPTIONS us on u.user_id = us.user_id
join dbo.SUBSCRIPTION_PLANS sp on us.plan_id = sp.plan_id
where sp.plan_name = 'Premium' and us.status = 'active';
```

```sql
select
u.user_id,
    u.username,
    u.email,
    us.end_date
from dbo.USERS u
join dbo.USER_SUBSCRIPTIONS us on u.user_id = us.user_id
join dbo.SUBSCRIPTION_PLANS sp on us.plan_id = sp.plan_id
where sp.plan_name = 'Premium' and us.status = 'active';
```

161 % No issues found

Results Messages

	user_id	username	email	end_date
1	1	user_1	user1@mail.com	2026-04-01
2	2	user_2	user2@mail.com	2026-05-15
3	5	user_5	user5@mail.com	2026-04-01
4	8	user_6	user6@mail.com	2026-05-01
5	10	user_8	user8@mail.com	2026-03-20
6	3	user_3	user3@mail.com	2026-06-01
7	11	user_9	user9@mail.com	2026-05-01
8	14	user_12	user12@mail.com	2026-06-01
9	16	user_14	user14@mail.com	2026-04-15
10	19	user_17	user17@mail.com	2026-05-01

Câu hỏi 23: Revenue tháng này

```sql
select
      sum(amount) as TotalRevenue,
      count(payment_id) as TotalTransactions,
      format(getdate(), 'MM/yyyy') as CurrentMonth
from dbo.PAYMENTS
where status = 'completed'
and month(payment_date) = month(getdate())
and year(payment_date) = year(getdate());
```

```sql
select
    sum(amount) as TotalRevenue,
    count(payment_id) as TotalTransactions,
    format(getdate(), 'MM/yyyy') as CurrentMonth
from dbo.PAYMENTS
where status = 'completed'
and month(payment_date) = month(getdate())
and year(payment_date) = year(getdate());
```

161 % No issues found

Results Messages

	TotalRevenue	TotalTransactions	CurrentMonth
1	82.91	9	03/2026

Câu hỏi 24: Subscription sắp hết hạn trong vòng 3 tháng

```sql
select
     u.username,
    u.email,
    sp.plan_name,
    us.end_date,
     datediff(month, getdate(), us.end_date) as MonthsLeft
from dbo.USER_SUBSCRIPTIONS us
join dbo.USERS u on us.user_id = u.user_id
join dbo.SUBSCRIPTION_PLANS sp on us.plan_id = sp.plan_id
where us.status = 'active'
and us.end_date >= getdate()
and us.end_date <= dateadd(month, 3, getdate())
order by us.end_date asc;
```

```sql
select
    u.username,
    u.email,
    sp.plan_name,
    us.end_date,
    datediff(month, getdate(), us.end_date) as MonthsLeft
from dbo.USER_SUBSCRIPTIONS us
join dbo.USERS u on us.user_id = u.user_id
join dbo.SUBSCRIPTION_PLANS sp on us.plan_id = sp.plan_id
where us.status = 'active'
and us.end_date >= getdate()
and us.end_date <= dateadd(month, 3, getdate())
order by us.end_date asc;
```

161 % No issues found

Results Messages

	username	email	plan_name	end_date	MonthsLeft
1	user_8	user8@mail.com	Premium	2026-03-20	0
2	mod_1	mod@mail.com	Basic	2026-03-25	0
3	user_1	user1@mail.com	Premium	2026-04-01	1
4	user_5	user5@mail.com	Premium	2026-04-01	1
5	admin_1	admin@mail.com	Free	2026-04-01	1
6	user_10	user10@mail.com	Basic	2026-04-01	1
7	user_15	user15@mail.com	Basic	2026-04-01	1
8	user_16	user16@mail.com	Free	2026-04-01	1

Câu hỏi 25: Lịch sử thanh toán của user 1

```sql
SELECT
    u.username,
    u.email,
    sp.plan_name,
    p.amount,
    p.payment_date,
    p.status AS payment_status,
    p.transaction_id
FROM USERS u
JOIN USER_SUBSCRIPTIONS us ON u.user_id = us.user_id
JOIN PAYMENTS p ON us.subscription_id = p.subscription_id
JOIN SUBSCRIPTION_PLANS sp ON us.plan_id = sp.plan_id
WHERE u.user_id = 1
ORDER BY p.payment_date DESC;
```

```sql
SELECT
    u.username,
    u.email,
    sp.plan_name,
    p.amount,
    p.payment_date,
    p.status AS payment_status,
    p.transaction_id
FROM USERS u
JOIN USER_SUBSCRIPTIONS us ON u.user_id = us.user_id
JOIN PAYMENTS p ON us.subscription_id = p.subscription_id
JOIN SUBSCRIPTION_PLANS sp ON us.plan_id = sp.plan_id
WHERE u.user_id = 1
ORDER BY p.payment_date DESC;
```

161 % No issues found

Results Messages

	username	email	plan_name	amount	payment_date	payment_status	transaction_id
1	user_1	user1@mail.com	Premium	9.99	2026-03-01 09:00:00.000	completed	TXN_004
2	user_1	user1@mail.com	Premium	9.99	2026-02-01 09:00:00.000	completed	TXN_003
3	user_1	user1@mail.com	Premium	9.99	2026-01-01 09:00:00.000	completed	TXN_002
4	user_1	user1@mail.com	Premium	9.99	2025-12-01 09:00:00.000	completed	TXN_001
5	user_1	user1@mail.com	Basic	4.99	2025-06-01 08:00:00.000	completed	TXN_018

Câu hỏi 26: Top 10 bài hát được nghe nhiều nhất

```sql
WITH PlayCount AS (
    SELECT
        song_id,
        COUNT(*) AS total_plays
    FROM USER_SONGS
    GROUP BY song_id
),
ArtistList AS (
    SELECT
        sa.song_id,
        STRING_AGG(ar.name, ', ') AS artists
    FROM SONG_ARTISTS sa
    JOIN ARTISTS ar ON sa.artist_id = ar.artist_id
    GROUP BY sa.song_id
)
SELECT TOP 10
    s.song_id,
    s.name AS song_name,
    a.title AS album_name,
    al.artists,
```

```sql
    pc.total_plays
FROM PlayCount pc
JOIN SONGS s ON pc.song_id = s.song_id
LEFT JOIN ALBUMS a ON s.album_id = a.album_id
LEFT JOIN ArtistList al ON s.song_id = al.song_id
ORDER BY pc.total_plays DESC;
```

```sql
WITH PlayCount AS (
    SELECT
        song_id,
        COUNT(*) AS total_plays
    FROM USER_SONGS
    GROUP BY song_id
),
ArtistList AS (
    SELECT
```

161 % No issues found

Results Messages

	song_id	song_name	album_name	artists	total_plays
1	1	Lạc Trôi	M-TP M-TP	Sơn Tùng M-TP	12
2	9	How You Like That	The Album	BLACKPINK	11
3	4	Hãy Trao Cho Anh	Sky Tour	Karik, Sơn Tùng M-TP	10
4	3	Chạy Ngay Đi	Sky Tour	Sơn Tùng M-TP	7
5	36	Dynamite	Map of the Soul: 7	BTS	7
6	17	Shape of You	÷ (Divide)	Ed Sheeran	6
7	20	Shake It Off	Lover	Taylor Swift	6
8	6	Nắng Ấm Xa Dần	M-TP M-TP	Sơn Tùng M-TP	6
9	38	24K Magic	24K Magic	Bruno Mars	5
10	26	Hãy Về Đây Bên Anh	Tâm 9	Mỹ Tâm	5

Câu hỏi 27: Bài hát trending (7 ngày qua)

```sql
WITH PlayCount AS (
    SELECT
        song_id,
        COUNT(*) AS total_plays
    FROM USER_SONGS
    WHERE listened_at >= DATEADD(DAY, -7, GETDATE())
    GROUP BY song_id
),
ArtistList AS (
    SELECT
        sa.song_id,
        STRING_AGG(ar.name, ', ') AS artists
```

```sql
    FROM SONG_ARTISTS sa
    JOIN ARTISTS ar
        ON sa.artist_id = ar.artist_id
    GROUP BY sa.song_id
)
SELECT TOP 10
    s.song_id,
    s.name AS song_name,
    a.title AS album_name,
    al.artists,
    pc.total_plays
FROM PlayCount pc
JOIN SONGS s
    ON pc.song_id = s.song_id
LEFT JOIN ALBUMS a
    ON s.album_id = a.album_id
LEFT JOIN ArtistList al
    ON s.song_id = al.song_id
ORDER BY pc.total_plays DESC;
```

```sql
WITH PlayCount AS (
    SELECT
        song_id,
        COUNT(*) AS total_plays
    FROM USER_SONGS
    WHERE listened_at >= DATEADD(DAY, -7, GETDATE())
    GROUP BY song_id
),
ArtistList AS (
```

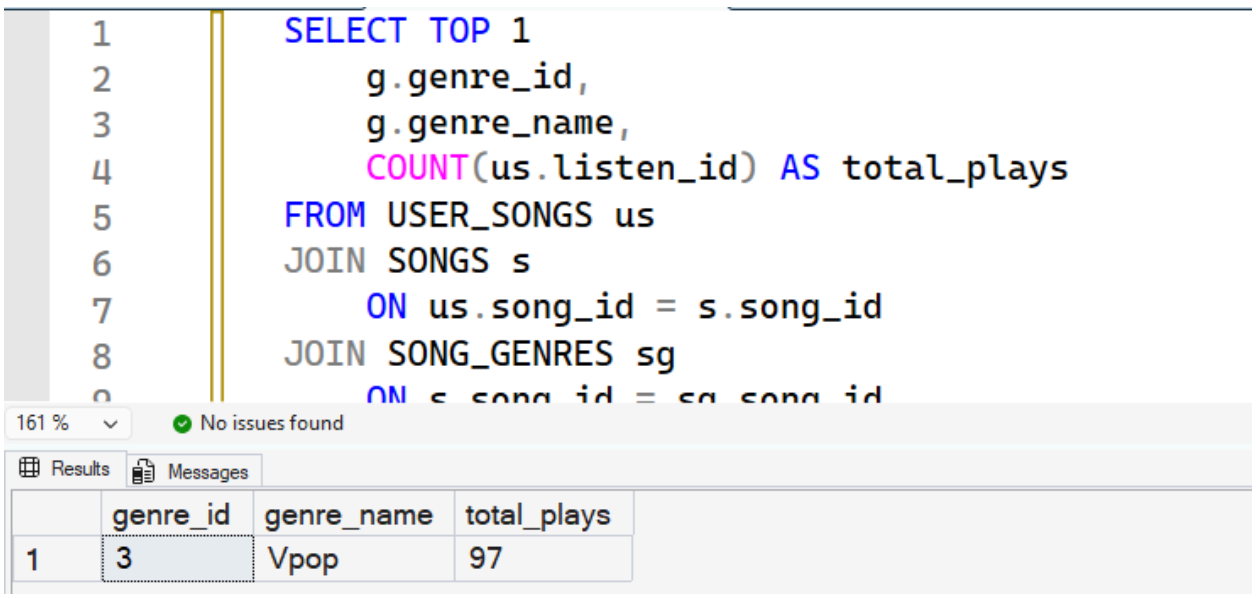
161 % No issues found

Results Messages

	song_id	song_name	album_name	artists	total_plays
1	1	Lạc Trôi	M-TP M-TP	Sơn Tùng M-TP	8
2	9	How You Like That	The Album	BLACKPINK	7
3	4	Hãy Trao Cho Anh	Sky Tour	Karik, Sơn Tùng M-TP	5
4	3	Chạy Ngay Đi	Sky Tour	Sơn Tùng M-TP	5
5	17	Shape of You	÷ (Divide)	Ed Sheeran	4
6	36	Dynamite	Map of the Soul: 7	BTS	4
7	7	Bài Này Chill Phết	Đông Kiếm Em	Đen Vâu	3
8	6	Nắng Ấm Xa Dần	M-TP M-TP	Sơn Tùng M-TP	3
9	22	Sóng Gió	Jack Universe	Karik, Jack	3
10	20	Shake It Off	Lover	Taylor Swift	3

Câu hỏi 28: Thể loại phổ biến nhất

```sql
SELECT TOP 1
    g.genre_id,
    g.genre_name,
    COUNT(us.listen_id) AS total_plays
FROM USER_SONGS us
JOIN SONGS s
    ON us.song_id = s.song_id
JOIN SONG_GENRES sg
    ON s.song_id = sg.song_id
JOIN GENRES g
    ON sg.genre_id = g.genre_id
GROUP BY g.genre_id, g.genre_name
ORDER BY total_plays DESC;
```



Câu hỏi 29: User nghe nhạc nhiều nhất

```sql
SELECT TOP 1
    u.user_id,
    u.username,
    COUNT(us.listen_id) AS total_listens
FROM USER_SONGS us
JOIN USERS u ON us.user_id = u.user_id
GROUP BY u.user_id, u.username
ORDER BY total_listens DESC;
```

```sql
SELECT TOP 1
    u.user_id,
    u.username,
    COUNT(us.listen_id) AS total_listens
FROM USER_SONGS us
JOIN USERS u ON us.user_id = u.user_id
GROUP BY u.user_id, u.username
ORDER BY total_listens DESC;
```

161 % No issues found

Results Messages

	user_id	username	total_listens
1	1	user_1	45

Câu hỏi 30: Top 10 bài hát được yêu thích nhất

```sql
SELECT TOP 10
    s.song_id,
    s.name AS song_name,
    ar.name AS artist_name,
    COUNT(sl.user_id) AS total_likes
FROM SONGS s
JOIN SONG_LIKES sl
    ON s.song_id = sl.song_id
LEFT JOIN SONG_ARTISTS sa
    ON s.song_id = sa.song_id
LEFT JOIN ARTISTS ar
    ON sa.artist_id = ar.artist_id
GROUP BY s.song_id, s.name, ar.name
ORDER BY total_likes DESC;
```

```sql
SELECT TOP 10
    s.song_id,
    s.name AS song_name,
    ar.name AS artist_name,
    COUNT(sl.user_id) AS total_likes
FROM SONGS s
JOIN SONG_LIKES sl
    ON s.song_id = sl.song_id
LEFT JOIN SONG_ARTISTS sa
```

161 % No issues found

Results | Messages

	song_id	song_name	artist_name	total_likes
1	9	How You Like That	BLACKPINK	5
2	1	Lạc Trôi	Sơn Tùng M-TP	5
3	4	Hãy Trao Cho Anh	Karik	4
4	4	Hãy Trao Cho Anh	Sơn Tùng M-TP	4
5	15	Rời Bỏ	Hòa Minzy	3
6	17	Shape of You	Ed Sheeran	3
7	42	Có Chàng Trai Viết Lên Cây	Phan Mạnh Quỳnh	3
8	3	Chạy Ngay Đi	Sơn Tùng M-TP	3
9	6	Nắng Ấm Xa Dần	Sơn Tùng M-TP	3
10	20	Shake It Off	Taylor Swift	3

Câu hỏi 31: Đăng ký người dùng mới (mặc định vai trò là user)

```sql
INSERT INTO USERS (username, email, password_hash, role_id)
VALUES (N'NguyenVanA', 'vana@mail.com', 'secure_hash_123', (SELECT
role_id FROM ROLES WHERE role_name = 'user'));
```

```sql
INSERT INTO USERS (username, email, password_hash, role_id)
VALUES (N'NguyenVanA', 'vana@mail.com', 'secure_hash_123', (SELECT role_id FROM ROLES WHERE role_name = 'user'));
```

161 % No issues found

Messages

```
(1 row affected)

Completion time: 2026-03-18T02:09:43.1709599+07:00
```

Câu hỏi 32: Người dùng tạo Playlist mới:

```sql
INSERT INTO PLAYLISTS (user_id, name, description, is_public)
VALUES (1, N'Nhạc Chill Cuối Tuần', N'Danh sách các bài hát thư
giãn', 1);
```

```sql
INSERT INTO PLAYLISTS (user_id, name, description, is_public)
VALUES (1, N'Nhạc Chill Cuối Tuần', N'Danh sách các bài hát thư giãn', 1);
```

161 % No issues found

Messages

```
(1 row affected)

Completion time: 2026-03-18T02:10:07.9286341+07:00
```

Câu hỏi 33: Thêm bài hát vào Playlist (Bảng trung gian):

```sql
INSERT INTO PLAYLIST_SONGS (playlist_id, song_id, position)
VALUES (1, 10, (SELECT ISNULL(MAX(position), 0) + 1 FROM
PLAYLIST_SONGS WHERE playlist_id = 1));
```

```sql
INSERT INTO PLAYLIST_SONGS (playlist_id, song_id, position)
VALUES (1, 10, (SELECT ISNULL(MAX(position), 0) + 1 FROM PLAYLIST_SONGS WHERE playlist_id = 1));
```

% No issues found

Messages

```
(1 row affected)

Completion time: 2026-03-18T02:10:31.9264121+07:00
```

Câu hỏi 34: Thực hiện thanh toán cho một gói đăng ký:

```sql
INSERT INTO PAYMENTS (subscription_id, amount, status,
transaction_id)
VALUES (5, 9.99, 'completed', 'TRANS_2026_XYZ');
```

```sql
INSERT INTO PAYMENTS (subscription_id, amount, status, transaction_id)
VALUES (5, 9.99, 'completed', 'TRANS_2026_XYZ');
```

161 % No issues found

Messages

```
(1 row affected)

Completion time: 2026-03-18T02:10:53.0658111+07:00
```

Câu hỏi 35: Cập nhật thông tin bài hát:

```sql
UPDATE SONGS
SET lyrics = N'Lời bài hát đã được cập nhật chính xác hơn...'
WHERE song_id = 15;
```

```sql
UPDATE SONGS
SET lyrics = N'Lời bài hát đã được cập nhật chính xác hơn...'
WHERE song_id = 15;
```

PRESS (SQL Server 17.0.1000 - LIGHTZGLS\SangVM)

161 % No issues found

Messages

```
(1 row affected)

Completion time: 2026-03-18T02:11:08.6038363+07:00
```

Câu 36: Gia hạn gói đăng ký

```sql
UPDATE USER_SUBSCRIPTIONS
SET end_date = DATEADD(MONTH, 1, end_date), auto_renew = 1
```

```
WHERE subscription_id = 10;
```

```
1  UPDATE USER_SUBSCRIPTIONS
2  SET end_date = DATEADD(MONTH, 1, end_date), auto_renew = 1
3  WHERE subscription_id = 10;
```

161 % No issues found

Messages

```
(1 row affected)

Completion time: 2026-03-18T02:11:24.6997366+07:00
```

Câu hỏi 37: Xóa một bài hát ra khỏi Playlist cụ thể:

```
DELETE FROM PLAYLIST_SONGS
WHERE playlist_id = 1 AND song_id = 10;
```

```
1  DELETE FROM PLAYLIST_SONGS
2  WHERE playlist_id = 1 AND song_id = 10;
```

161 % No issues found

Messages

```
(1 row affected)

Completion time: 2026-03-18T02:11:41.2028628+07:00
```

Câu hỏi 38: Xóa một bình luận (Chỉ admin hoặc chủ sở hữu):

```
DELETE FROM SONG_COMMENTS WHERE comment_id = 100 AND user_id = 5;
```

```
1  DELETE FROM SONG_COMMENTS WHERE comment_id = 100 AND user_id = 5;
```

161 % No issues found

Messages

```
(1 row affected)

Completion time: 2026-03-18T02:16:52.4852423+07:00
```

Câu hỏi 39: Người dùng hủy lượt thích bài hát:

```
DELETE FROM SONG_LIKES
WHERE user_id = 5 AND song_id = 20;
```

```sql
DELETE FROM SONG_LIKES
WHERE user_id = 5 AND song_id = 20;
```

161 % No issues found

Messages

```
(1 row affected)

Completion time: 2026-03-18T02:16:52.4852423+07:00
```

Câu hỏi 40: Xóa toàn bộ Playlist của người dùng:

```sql
DELETE FROM PLAYLISTS WHERE playlist_id = 5;
```

```sql
DELETE FROM PLAYLISTS WHERE playlist_id = 5;
```

% No issues found

Messages

```
(1 row affected)

Completion time: 2026-03-18T02:17:30.4182786+07:00
```

Câu hỏi trên lớp

Câu 1: Bài hát được nghe nhiều nhất

Tên người viết câu truy vấn: **Đỗ Tiến Thành**

Mã SQL:

```sql
WITH PlayCount AS (
    SELECT
        song_id,
        COUNT(*) AS total_plays
    FROM USER_SONGS
    GROUP BY song_id
),
ArtistList AS (
    SELECT
        song_id,
        STRING_AGG(name, ', ') AS artists
    FROM (
```

```sql
        SELECT DISTINCT
            sa.song_id,
            ar.name
        FROM SONG_ARTISTS sa
        JOIN ARTISTS ar ON sa.artist_id = ar.artist_id
    ) AS DistinctArtists
    GROUP BY song_id
)
SELECT TOP 1
    s.name AS song_name,
    a.title AS album_name,
    al.artists,
    pc.total_plays
FROM PlayCount pc
JOIN SONGS s ON pc.song_id = s.song_id
LEFT JOIN ALBUMS a ON s.album_id = a.album_id
JOIN ArtistList al ON s.song_id = al.song_id
ORDER BY pc.total_plays DESC;
```

Kết quả:

136 %

Results | Messages

	song_name	album_name	artists	total_plays
1	See Tình	Hoàng	Hoàng Thùy Linh	37

Câu 2: Tháng có lượt nghe nhiều nhất

Tên người viết câu truy vấn: **An Việt Quyền**

Mã SQL:

```sql
SELECT TOP 12
    MONTH(listened_at) AS Thang,
    YEAR(listened_at) AS Nam,
    COUNT(listen_id) AS TongLuotNghe
FROM USER_SONGS
GROUP BY YEAR(listened_at), MONTH(listened_at)
```

```
ORDER BY TongLuotNghe DESC;
```

Kết quả:

Results | Messages

	Thang	Nam	TongLuotNghe
1	1	2026	13986
2	12	2025	13865
3	7	2025	13843
4	5	2025	13766
5	1	2025	13755
6	8	2025	13749
7	10	2025	13707
8	3	2025	13635
9	9	2025	13381
10	4	2025	13363
11	11	2025	13240
12	6	2025	13238

Câu 3: Bài hát có lượt nghe nhiều nhất trong tháng

Tên người viết câu truy vấn: **Vũ Minh Sáng**

Trường hợp 1: Tháng X xác định

```
SELECT TOP 1
    s.name,
    COUNT(us.listen_id) AS total_listens
FROM USER_SONGS us
JOIN SONGS s ON us.song_id = s.song_id
WHERE MONTH(us.listened_at) = 3 AND YEAR(us.listened_at) = 2026
GROUP BY s.song_id, s.name
ORDER BY total_listens DESC;
```

Mã SQL:

```sql
SELECT TOP 1
    s.name,
    COUNT(us.listen_id) AS total_listens
FROM USER_SONGS us
JOIN SONGS s ON us.song_id = s.song_id
WHERE MONTH(us.listened_at) = 3 AND YEAR(us.listened_at) = 2026
GROUP BY s.song_id, s.name
ORDER BY total_listens DESC;
```

176 % No issues found

Results Messages

	name	total_listens
1	See Tình	30

Kết quả:

176 % No issues found

Results Messages

	name	total_listens
1	See Tình	30

Trường hợp 2: Liệt kê theo tháng

Mã SQL:

```sql
SELECT TOP 1
    MONTH(us.listened_at) AS [Month],
    YEAR(us.listened_at) AS [Year],
    s.name,
    COUNT(us.listen_id) AS total_listens
FROM USER_SONGS us
JOIN SONGS s ON us.song_id = s.song_id
WHERE MONTH(us.listened_at) = MONTH(GETDATE())
  AND YEAR(us.listened_at) = YEAR(GETDATE())
GROUP BY MONTH(us.listened_at), YEAR(us.listened_at), s.name
ORDER BY total_listens DESC;
```

176 % No issues found

Results Messages

	Month	Year	name	total_listens
1	3	2026	See Tình	30

Câu 4: Liệt kê 5 sp đc nghe nhiều nhất mỗi tháng sắp xếp theo tháng

Tên người viết câu truy vấn: **Vũ Việt Thái**

Mã SQL:

```
select
      rs.MonthYear,
      s.name as SongName,
      rs.total_plays
from (
      select
            MonthYear,
            song_id,
            total_plays,
            row_number() over (
                  partition by MonthYear
                  order by total_plays desc
            ) as rn
      from (
            select
                  format(listened_at, 'MM/yyyy') as MonthYear,
                  song_id,
                  count (*) as total_plays
            from dbo.USER_SONGS
            group by format(listened_at, 'MM/yyyy'), song_id
      ) as _
) as rs
join dbo.SONGS s on rs.song_id = s.song_id
where rs.rn <= 5
order by cast('01/' + rs.MonthYear AS DATE) asc;
```

Kết quả:

100 % No issues found

Results Messages

	MonthYear	SongName	total_plays
1	12/2024	Song Name 512	2
2	12/2024	Song Name 73067	2
3	12/2024	Song Name 71081	2
4	12/2024	Song Name 70772	2
5	12/2024	Song Name 57832	2
6	01/2025	Song Name 79743	3
7	01/2025	Song Name 22211	3
8	01/2025	Song Name 86407	3
9	01/2025	Song Name 12158	3
10	01/2025	Song Name 71783	3
11	02/2025	Song Name 34832	3
12	02/2025	Song Name 33565	3
13	02/2025	Song Name 24900	2
14	02/2025	Song Name 11693	2
15	02/2025	Song Name 1142	2
16	03/2025	Song Name 92230	3
17	03/2025	Song Name 56603	3
18	03/2025	Song Name 58581	3
19	03/2025	Song Name 56720	3
20	03/2025	Song Name 49715	3

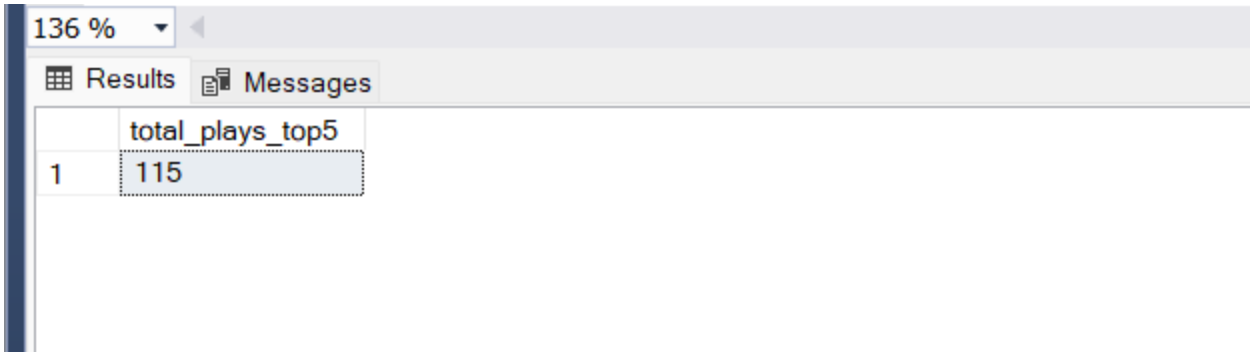
Câu 5: Tính tổng số lượt nghe của 5 sp được nghe nhiều nhất

Tên người viết câu truy vấn: **Đỗ Tiến Thành**

Mã SQL:

```
WITH PlayCount AS (
    SELECT TOP 5
        song_id,
        COUNT(*) AS total_plays
    FROM USER_SONGS
    GROUP BY song_id
      ORDER BY total_plays DESC
)
select
      sum(total_plays) as top5_total_plays
from PlayCount;
```

Kết quả:



VIII. Tối ưu hóa hiệu năng truy vấn với Chỉ mục (Indexing)

Trong hệ thống cơ sở dữ liệu của ứng dụng nghe nhạc, mặc định hệ quản trị đã tự động tạo **Clustered Index** cho tất cả các Khóa chính như user_id, song_id, playlist_id, v.v. Điều này giúp tối ưu hóa việc tìm kiếm trực tiếp qua khóa chính.

Tuy nhiên, với các truy vấn nghiệp vụ phức tạp ở phần VII, nếu chỉ phụ thuộc vào khóa chính, hệ thống sẽ gặp hiện tượng "nút thắt cổ chai" khi khối lượng dữ liệu tăng lên hàng triệu bản ghi. Để giải quyết, nhóm tiến hành phân tích các kịch bản thực tế và đề xuất tạo **Non-Clustered Index** như sau:

1. Kịch bản 1: Tìm kiếm dữ liệu văn bản (Text Search)

- **Câu hỏi thực tế (Câu 2):** Tìm bài hát theo tên (Ví dụ: SELECT * FROM SONGS WHERE name LIKE N'%Nắng%';).
- **Vấn đề :** Bảng SONGS là bảng cốt lõi chứa lượng lớn dữ liệu. Khi sử dụng mệnh đề WHERE trên cột name không có chỉ mục, hệ quản trị bắt buộc phải thực hiện quét toàn bộ bảng (Table Scan / Clustered Index Scan) để đối chiếu từng chuỗi. Quá trình này tiêu tốn rất nhiều tài nguyên I/O và thời gian xử lý.

Giải pháp đánh Index: Tạo một Non-Clustered Index trên cột name.

```
CREATE NONCLUSTERED INDEX IX_Songs_Name ON SONGS(name);
```

- *Đánh giá:* Mặc dù với toán tử LIKE '%...', Index B-Tree có thể không phát huy tối đa 100% sức mạnh so với Full-Text Search, nhưng nó cải thiện đáng kể tốc độ cho các truy vấn tìm kiếm chính xác (=) hoặc tìm kiếm tiền tố (ví dụ: LIKE N'Nắng%').

2. Kịch bản 2: Truy vấn thống kê trên tập dữ liệu lớn (Aggregation)

- **Câu hỏi thực tế (Câu 26):** Thống kê Top 10 bài hát được nghe nhiều nhất. Truy vấn sử dụng GROUP BY song_id và hàm COUNT(*) trên bảng USER_SONGS.

- **Vấn đề:** Bảng USER_SONGS ghi nhận lịch sử nghe nhạc của người dùng nên dữ liệu phát sinh liên tục theo cấp số nhân. Việc gom nhóm (GROUP BY) và đếm lượt nghe yêu cầu hệ thống đọc toàn bộ các trang dữ liệu vật lý của bảng này.

Giải pháp đánh Index: Tạo Non-Clustered Index trên cột song_id. Khi đó, hệ quản trị chỉ cần duyệt qua cây chỉ mục (đã được sắp xếp sẵn theo song_id) để đếm số lượng bản ghi, thay vì phải truy xuất vào dữ liệu vật lý của toàn bộ bảng.

```
CREATE NONCLUSTERED INDEX IX_UserSongs_SongId ON USER_SONGS(song_id);
```

3. Kịch bản 3: Truy vấn kết hợp nhiều điều kiện lọc

- **Câu hỏi thực tế (Câu 24):** Tìm các gói đăng ký (Subscription) sắp hết hạn trong vòng 3 tháng với điều kiện status = 'active' và end_date <= DATEADD(month, 3, GETDATE()).
- **Vấn đề (Bottleneck):** Bảng USER_SUBSCRIPTIONS chứa cả những gói đang hoạt động, đã hủy (cancelled) hoặc đã hết hạn (expired). Nếu không có chỉ mục, engine phải kiểm tra từng dòng một để xem gói đó có thỏa mãn cả hai điều kiện hay không.

Giải pháp đánh Index: Sử dụng **Chỉ mục phức hợp** trên cả 2 cột status và end_date. Thứ tự cột được tối ưu bằng cách đặt cột lọc chính xác (status dùng toán tử =) lên trước, và cột lọc khoảng giá trị (end_date dùng toán tử <, >) ở sau.

```
CREATE NONCLUSTERED INDEX IX_UserSubs_Status_EndDate
ON USER_SUBSCRIPTIONS(status, end_date);
```

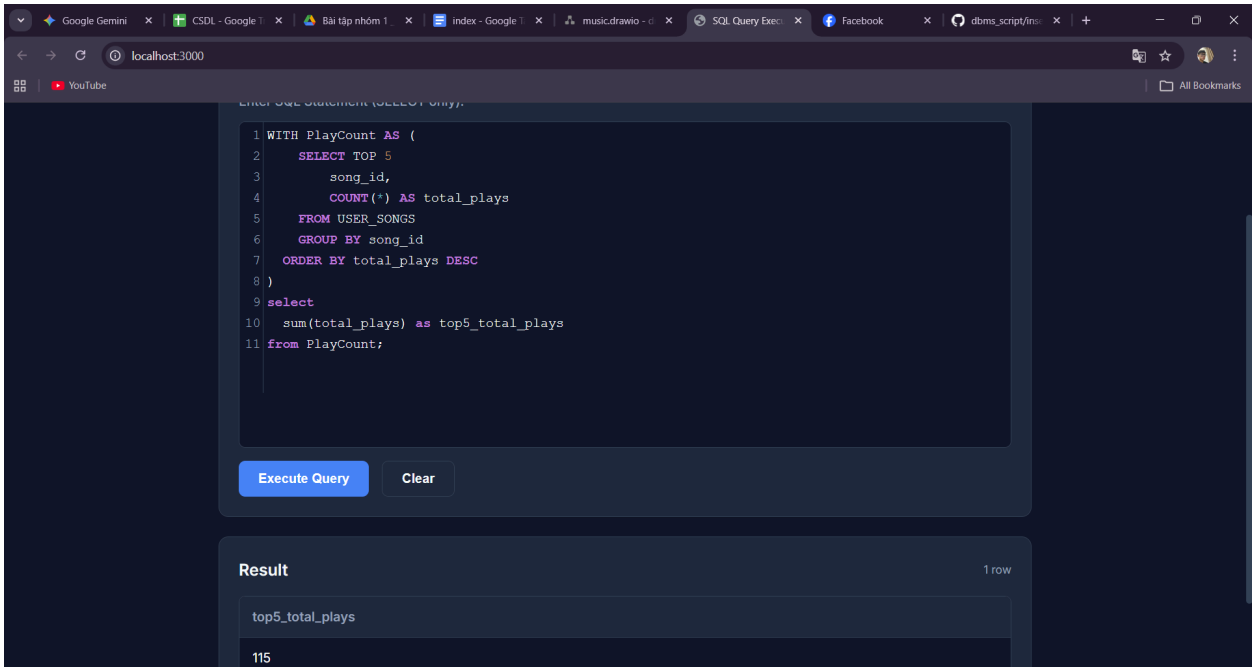
4. Kịch bản 4: Tối ưu hóa truy vấn kết hợp sắp xếp

- **Câu hỏi thực tế:** Lấy danh sách bình luận của một bài hát cụ thể và sắp xếp mới nhất (Ví dụ: WHERE song_id = 20 ORDER BY created_at DESC).
- **Vấn đề:** Bảng SONG_COMMENTS có khối lượng dữ liệu khổng lồ. Hệ thống không những phải quét tìm toàn bộ bình luận của song_id = 20 mà còn phải cấp phát bộ nhớ để thực hiện thuật toán sắp xếp theo thời gian.

Giải pháp đánh Index: Tạo một Composite Index bao phủ cả trường lọc và trường sắp xếp. Cây chỉ mục sẽ lưu trữ các bản ghi của cùng một bài hát theo thứ tự thời gian giảm dần từ trước, triệt tiêu hoàn toàn chi phí cho bước Sort.

```
CREATE NONCLUSTERED INDEX IX_SongComments_SongId_CreatedAt
ON SONG_COMMENTS(song_id, created_at DESC);
```

IX. Trang Web



X. Kết luận

Thông qua quá trình nghiên cứu và thực hiện đề tài, dự án đã hoàn thành việc xây dựng một hệ thống cơ sở dữ liệu hoàn chỉnh và mở rộng cho ứng dụng nghe nhạc online. Những kết quả chính đạt được bao gồm:

1. **Thiết kế logic vững chắc**: Hệ thống đã bao quát đầy đủ các thực thể cốt lõi như Người dùng (với phân quyền vai trò), Bài hát, Nghệ sĩ, Album, Thể loại, Playlist và các tính năng tương tác (bình luận, lượt thích), đồng thời phản ánh chính xác các quy tắc nghiệp vụ thực tế. Bổ sung thêm hệ thống quản lý gói đăng ký và thanh toán so với thiết kế cơ bản.

2. **Tối ưu hóa dữ liệu**: Mô hình quan hệ đã được chuẩn hóa đạt dạng chuẩn BCNF, giúp loại bỏ tối đa sự dư thừa dữ liệu và đảm bảo tính toàn vẹn thông qua các ràng buộc chặt chẽ (Primary Key, Foreign Key, Check, Unique, Default). Sử dụng 5 bảng trung gian để xử lý các quan hệ N-N.

3. **Tính thực tiễn cao**: Các thử nghiệm truy vấn SQL (như hệ thống đề xuất, thống kê bài hát thịnh hành, quản lý playlist, lịch sử nghe) đã chứng minh khả năng đáp ứng tốt các tính năng quan trọng của một ứng dụng âm nhạc hiện đại, từ quản lý tài nguyên đến cá nhân hóa trải nghiệm người dùng.

Tổng thể, cơ sở dữ liệu này cung cấp một nền tảng ổn định và hiệu quả với **17 bảng** được thiết kế đầy đủ, sẵn sàng để tích hợp vào các ứng dụng thực tế hoặc mở rộng thêm các tính năng nâng cao.

Tài liệu tham khảo

1. Học viện Công nghệ Bưu chính Viễn thông, Slide bài giảng môn Hệ quản trị Cơ sở dữ liệu, Khoa Công nghệ Thông tin.

2. Đỗ Trung Tuấn, Giáo trình Cơ sở dữ liệu, NXB Giáo dục.

3. Nguyễn Kim Anh (2007), Giáo trình Hệ quản trị Cơ sở dữ liệu, Nhà xuất bản Giáo dục.

4. Microsoft, SQL Server Technical Documentation, SQL Server Technical Documentation.

5. Abraham Silberschatz, Henry F. Korth, S. Sudarshan (2019), Database System Concepts (7th Edition), McGraw-Hill Education.